目录

0x00.多项式基本概述.....2
0x01. 多项式的基础概念.....2
0x02. 多项式的度.....3
0x03. 多项式的乘法.....3
0x04. 多项式的逆元.....4
0x05. 多项式多点求值与插值.....4
0x06 卷积的定义.....4
0x07 卷积的基本性质.....5
0x10.多项式插值.....5
0x11.拉格朗日插值.....5
0x11.1 拉格朗日插值简述.....6
0x11.2 拉格朗日插值模板.....6
0x11.3 拉格朗日插值法求系数.....7
0x12.牛顿插值.....8
0x20.多项式乘法.....8
0x21.快速傅里叶变换（FFT）.....8
0x21.1 FFT 简述.....8
0x21.2. 递归版.....10
0x21.3. 迭代版（蝴蝶变换）.....11
0x21.4. “三步变两步”优化.....13
0x22.快速数论变换（NTT）.....13
0x22.1NTT简述.....14
0x23.快速沃尔什变换（FWT）.....16
0x24.快速莫比乌斯变换（FMT）.....16
0x25.任意模数变换（MTT）.....16
0x30.多项式求逆.....17
0x31. 多项式求逆简述.....17
0x32. 多项式求逆模板.....18
0x40.多项式开方.....20
0x41. 多项式开方简述.....20
0x42. 多项式开方(根)模板.....20
0x50.多项式除法 / 取模.....21
0x51.多项式除法简述.....22
0x52. 多项式除法模板.....22
0x60.多项式牛顿迭代.....24
0x61. 多项式牛顿迭代及其应用.....24
0x70.多项式多点求值 / 快速插值.....25
0x80.多项式三角函数.....25
0x90.多项式反三角函数.....26
0x100.常系数齐次线性递推（多项式优化）.....26
0x110.快速差分/前缀和.....26
0x120.多项式对数（多项式求导 + 积分）.....26
0x121. 多项式对数简述（含常见函数求导表）.....26
0x122. 多项式对数函数（多项式 ln）模板.....27
0x130.多项式指数（大毒瘤 ）.....29
0x140.多项式快速幂.....29
0x150.多项式全家桶！！！（code）.....29

名称	公式	解决方法	复杂度
多项式乘法	$F(x)=A(x)B(x)$	NTT/FFT	$O(n\log n)$
多项式求逆	$B(x)\equiv 2B_1(x)-B_1^2(x)A(x)(\mod x^n)$	分治+多项式乘法	$O(n\log n)$
多项式除法	$A^R(x)B^{R^{-1}}(x)\equiv D^R(x)(\mod x^{n-m+1})$	多项式求逆	$O(n\log n)$
多项式取模	$P(x)=A(x)-D(x)B(x)$	多项式乘法	$O(n\log n)$
多项式求导	$F'(x)=\sum_{i=1}^{n}ia_ix^{i-1}$直接算	——	$O(n)$
多项式积分	$\int F(x)=\sum_{i=1}^{n}\frac{a_ix^{i+1}}{i+1}$直接算	——	$O(n)$
多项式开根	$B(x)\equiv\frac{B_1^2(x)+A(x)}{2B_1(x)}(\mod x^n)$	分治+多项式求逆	$O(n\log n)$
多项式ln	$ln(F(x))=\int F'(x)F^{-1}(x)$	多项式求逆	$O(n\log n)$
多项式exp	$F(x)\equiv F_0(x)(1-\ln F_0(x)+A(x))(\mod x^n)$	多项式ln	$O(n\log n)$

https://blog.csdn.net/weixin_45697774

tips:

> 注意一点，在我们每次求 `limit` 的时候，把范围都乘 `2` ，反正乘了开大一点不会错，最多会跑的慢一点，但是有时候不开就会 `WA` （比如在求**多项式除法**的时候）
> `limit` 可以设成全局变量，注意每次使用 `limit` 的时候都要按照下面的格式初始化一下 `limit` 以及 `L`
>
> @ **FZzzz** 2020-06-30 18:29 回复
>
> 求逆在 $\mod x^{n-m+1}$ 下做，但乘法时 FFT 长度要开到 $2n-2m+1$ 向上第一个二的次幂
>
> 就是这里：

```
for(limit = 1, L = 0; limit ≤ (n + m) * 2; limit <<= 1) L ++ ;
for(int i = 0; i < limit; ++ i)
    RR[i] = (RR[i >> 1] >> 1) | ((i & 1) << (L - 1));
```

0x00.多项式基本概述

既然是多项式全家桶，那么肯定要先来了解一下什么是多项式hhh，基础一定要扎实，不然到后面让你怀疑人生

0x01. 多项式的基础概念

首先是多项式的一些基础的概念：

多项式：几个单项式的和（单项式是由数字或字母组成的积的代数和）

本质上就是才满足一些运算性质的环

例：$x^2+2x-15$，$p(x)=a_0+a_1x+a_2x^2+\cdots+a_nx^n$　多项式 ≠ 函数

给定一个域 k，定义一个环 $k[x]$，称为 k 上的一元多项式环，$k[x]$ 中任一元素称为多项式

$k[x]=\{a_0+a_1x+a_2x^2+\cdots a_nx^n \mid n$ 是自然数，$a_0,a_1,\cdots a_n\in k\}$

- 多项式 + 多项式 = 新的多项式
- 多项式 × 多项式 = 新的多项式

环仅依赖于域 k，x 仅为一个记号，可替换为其它字母

一个多项式可以表示为一个有限长数组 $(a_0,a_1,\cdots a_n)$

或者一个只有有限项非零的无限长数组 $(a_0,a_1,\cdots a_n,0,0,0\cdots)$

多项式的零点个数是有限的，非多项式的零点个数可以是无穷多个

例：$\sin x$ 是一个无穷多项展开的展开式，$\sin x$ 的展开式不是一个多项式

- 系数表示法

  设 $A(x)$ 表示一个 $n-1$ 次多项式。

  则 $A(x)=\sum_{i=0}^{n} a_i*x^i$

  例：$A(3)=2+3*x+x^2$

  求多项式乘法时间复杂度 $O(n^2)$（两个多项式的每个系数两两相乘）

- 点值表示法

  将 n 个互不相同的 x 带入多项式，得到 n 个不同的取值 y。

  则该多项式被这 n 个点：$(x_1,y_1),(x_2,y_2),\cdots(x_n,y_n)$ 唯一确定

  定理1：一个 $n-1$ 次多项式在 n 个不同点的取值唯一确定这个多项式

  其中 $y_i=\sum_{j=0}^{n-1} a_j*x_i^j$

  例 $A(3)=2+3*x+x^2 \Rightarrow (0,2),(1,6),(2,16)$

  求多项式乘法时间复杂度 $O(n^2)$（$O(n)$ 选点，每次 $O(n)$ 计算）

https://blog.csdn.net/weixin_45697774

0x02. 多项式的度

对于一个多项式 $f(x)$，称其最高次项的次数为该多项式的 度（Degree），记作 $deg\ f$。

好吧就是多项式的次数。（顺带一提，单项式的次数是所有字母次数之和hhh）

0x03. 多项式的乘法

我们上面也说了多项式 × 多项式 = 新的多项式。

所以我们最关心的操作就是两个多项式的乘法 —— 卷积。

广义上的卷积为：

$$h(x)=\int_{-\infty}^{\infty}g(\tau)\cdot f(x-\tau)\mathrm{d}\tau \tag{1}$$

我们这里仅讨论多项式域，即两个多项式 / 数组 / 序列的卷积。

即给定两个多项式 $f(x)$，$g(x)$：

$f(x)=a_0+a_1x+\cdots+a_nx^n$

$g(x)=b_0+b_1x+\cdots+b_mx^m$

计算多项式 $Q(x)=f(x)\ \cdot\ g(x)$：

$$\boxed{Q(x)=\sum_{i=0}^{n}\sum_{j=0}^{m}a_ib_jx^{i+j}}=c_0+c_1x+\cdots+c_{n+m}x^{n+m}$$

$$C(x)=A(x)*B(x)=\sum_{i=0}^{n}(\sum_{j=0}^{i}a_jb_{i-j})x^{j+i-j}=\sum_{i=0}^{n}(\sum_{j=0}^{i}a_jb_{i-j})x^i \tag{2}$$

也就是对于 $C(x)$ 的第 i 项的系数：

$$[i]C(x)=\sum_{j=0}^{i}a_jb_{i-j} \tag{3}$$

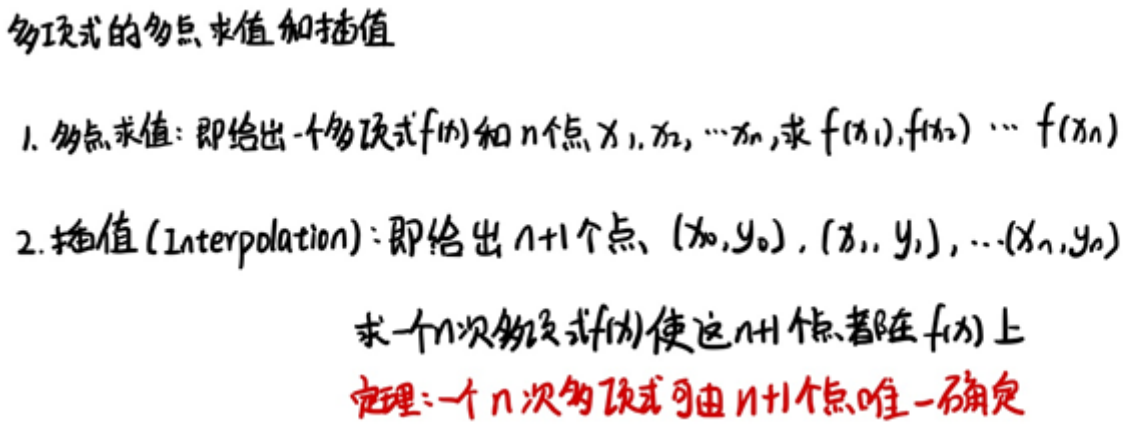
$$C(x)=A(x)*B(x)=\sum_{k=0}^{n+m-2}(\sum_{k=i+j}a_ib_j)x^k \tag{4}$$

这里使用的就是上面介绍的系数表示，我们如果直接暴力相乘很明显是一个 $O(nm)$ 的复杂度，因为我们每一项都需要相乘。但是我们可以通过快速傅里叶变换将常用的系数表示转为点值表示用 $O(nlogn)$ 的时间复杂度下计算得到答案，再逆变换转成系数表示输出，将会在下面章节介绍。

0x04. 多项式的逆元

对于 n 次多项式$f(x)$，若存在$g(x)$，满足：
$f(x)g(x)\equiv 1\ \pmod{x^n}\ \&\&\ \deg g\le\deg f$
则称$g(x)$是 n 次多项式 $f(x)$ 的逆元。

0x05. 多项式多点求值与插值



0x06 卷积的定义

对于一个序列，将其中元素一一映射到一个多项式函数的系数上， 这个多项式函数便叫做该序列的**生成函数**。

形式化地讲，对于序列 $f_0,f_1,\cdots,f_{n-1}$ ， $f(x)=\sum_{k=0}^{n-1}f_kx^k$ 为其生成函数。

卷积即为生成函数的乘积在对应序列的变换上的的抽象，“卷”即为其作用效果，“积”即为其本质。

对于序列 f,g，其卷积序列 $f\otimes g$ 满足 $(f\otimes g)_k=\sum\limits_{i=0}^{k}f_i\times g_{k-i}=\sum\limits_{i,j}^{i+j=k}f_i\times g_j$

对于多项式 f,g，其多项式的卷积为：$f\otimes g=\sum\limits_{k=0}^{n}(\sum\limits_{i,j}^{i+j=k}a_i\times b_j)x^k$

而我们所熟知的 FFT 计算的是循环卷积，也即 $(f\otimes g)_k=\sum\limits_{i+j\equiv k\pmod n}f_i\times g_j$，$n$ 为序列长度。

0x07 卷积的基本性质

这里的卷积均为序列的卷积，因此证明时我们使用数学归纳法，即证明第 k 项成立，则整个序列均成立。由于多项式实际上就是序列的生成函数，所以性质同样成立。

- $f\otimes g=g\otimes f$（交换律）
  我们使用定义 $(f\otimes g)_k=\sum\limits_{i+j=k}f_i\times g_j$，以及乘法交换律 $a\times b=b\times a$ 即可证明，因为 $i+j=k$ 的 i 和 j 是一一对应的。

- $(f\otimes g)\otimes h=f\otimes(g\otimes h)$ （结合律）
  证明：

$$
\begin{aligned}
[f\otimes(g\otimes h)]_n&=\sum_{i=0}^{n}f_{n-i}\times(g\otimes h)_i\\
&=\sum_{i=0}^{n}f_{n-i}\sum_{j=0}^{i}g_jh_{i-j}\\
&=\sum_{i=0}^{n}\sum_{j=0}^{i}f_{n-i}g_jh_{i-j}\\
&=\sum_{i+j+k=n}f_ig_jh_k
\end{aligned}
\tag{5}
$$

交换顺序后反向推导即可得证。

- $(f\oplus g)\otimes h=(f\otimes h)\oplus(g\otimes h)$ （分配律）
  其中 $(f\oplus g)_k=af_k+bg_k$，即序列 f,g 的加法，a,b 为常数。

证明：

$$
\begin{aligned}
[(f\oplus g)\otimes h]_k&=\sum_{i=0}^{k}(f\oplus g)_ih_{k-i}\\
&=\sum_{i=0}^{k}(af_i+bg_i)h_{k-i}\\
&=\sum_{i=0}^{k}(af_ih_{k-i}+bg_ih_{k-i})\\
&=a\sum_{i=0}^{k}f_ih_{k-i}+b\sum_{i=0}^{k}g_ih_{k-i}\\
&=a(f\otimes h)_k+b(g\otimes h)_k=[(f\otimes h)\oplus(g\otimes h)]_k
\end{aligned}
\tag{6}
$$

0x10.多项式插值

0x11.拉格朗日插值

- （子博客）链接：**【学习笔记】拉格朗日插值** 包含全套证明，这里只说一些重点，具体的证明在链接里

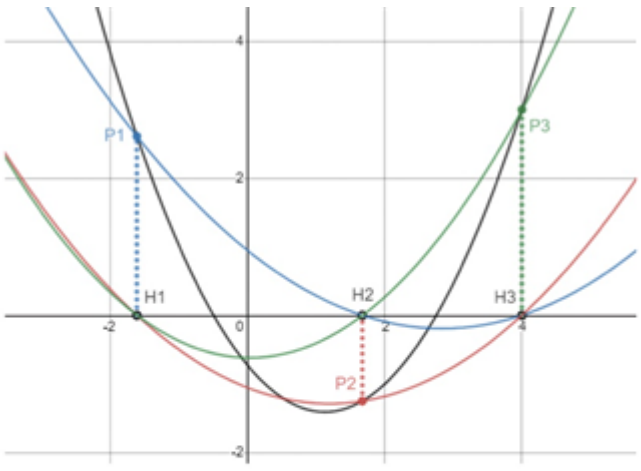
0x11.1 拉格朗日插值简述

定理：$n+1$ 个点可以唯一确定一个n 次多项式 $f(x)$。

现在给出n个点(x_i,y_i)，请确定多项式$f(k)\ mod\ 988244535$的值。

方法3：拉格朗日插值法

如右图，将n个点(x_i,y_i)在X轴上投影$(x_i,0)$记为H_i，对每个i，我们选择一个点集$\{P_i\}\cup\{H_j\mid 1\le i\le n,j\ne i\}$（其中$P_i$为给定的n个点），在这个点集中，作过这n个点的至多n-1次多项式$g_i(x)$。（$f(x)$黑线，$g_i(x)$彩线）



我们得到了n个$g_i(x)$（$1\le i\le n$），它们在各自对应的x_i处值为y_i（$g_i(x)$过P_i和$H_j(\forall j\ne i)$点，在其它$x_j(\forall j\ne i)$处值为0），易构造出$g_i(x)$的表达式

$$g_i(x)=y_i\prod_{j\ne i}\frac{x-x_j}{x_i-x_j}$$

类比两点式

$$\frac{x-x_1}{x_2-x_1}=\frac{y-y_1}{y_2-y_1}$$，对于$(x_i,y_i),(x_j,0)(\forall j\ne i)$

$$\frac{y-0}{y_i-0}=\frac{x-x_1}{x_2-x_1}\Rightarrow y=y_i\left(\frac{x-x_1}{x_2-x_1}\right)$$

我们所求$f(x)=\sum_{i=1}^{n}g_i(x)$，即为$g_i(x)$的和。（每一个i，都只有一条$g_i$过$P_i$，其余的有$g_i$均过$H_i$，故它们相加后在$x_i$的取值仍为$y_i$，即所有和函数总是经过所有$P_i$）

得$f(x)=\sum_{i=1}^{n}y_i\prod_{j\ne i}\frac{x-x_j}{x_i-x_j}$

若求每一次系数，$O(n^2)$但本题仅求$f(k)$即可，故直接代入k即可

$$f(k)=\sum_{i=1}^{n}y_i\prod_{j\ne i}\frac{k-x_j}{x_i-x_j}$$

$O(n^2)$

0x11.2 拉格朗日插值模板

我们按照上面的思路，带入n个点求一下$f(k)$即可。时间复杂度$O(n^2)$。

注意本题还要求逆元，为了防止求逆元的时间复杂度影响整体的时间复杂度，所以我们分别计算出分子和分母，再将分子乘进分母的逆元，累加进最后的答案，时间复杂度的瓶颈就不会在求逆元上，总体的时间复杂度为$O(n^2)$。

代码实现：

本题需要累乘，会爆int，记得开long long

（~~数学题开long long是个好习惯~~ ）

```cpp
const int mod = 998244353;
ll n, m, k;
struct Point
{
    ll x, y;
}A[N];
ll inv(ll x) {return qpow(x, mod - 2, mod);}//快速幂的代码我就删了省空间，应该都会

int main()
{
    scanf("%lld%lld", &n, &k);
    for(int i = 1; i ≤ n; ++ i) {
        scanf("%lld%lld", &A[i].x, &A[i].y);
    }
    ll ans = 0;
```

```

    for(int i = 1; i ≤ n; ++ i) {
        ll s1 = A[i].y % mod;
        ll s2 = 1ll;
        for(int j = 1; j ≤ n; ++ j) {
            if(i ≠ j) {
                s1 = s1 * (k - A[j].x) % mod;
                s2 = s2 * (A[i].x - A[j].x) % mod;
            }
        }
        ans += s1 * inv(s2) % mod;
    }
    printf("%lld\n", (ans % mod + mod) % mod);
    return 0;
}
```

我们使用拉格朗日插值公式，对于一个数 k，我们很容易在 $O(n^2)$ 时间求得 $F(k)$ 的数值，如果 x_i 是连续的，我们甚至可以利用预处理在 $O(n)$ 时间内得到 $F(k)$ 的数值。
但是如果 x_i 不连续，又有多组查询，就需要得到这个多项式的**系数**以保证求一个函数值的时间为 $O(n)$ 。

0x11.3 拉格朗日插值法求系数

首先观察式子，发现对于每个 i，上面部分是 $\frac{(x-x_1)\times(x-x_2)\ldots\times(x-x_n)}{(x-x_i)}$，下面那部分和 y_i 都是常数。

所以可以 $O(n^2)$ 处理出$(x-x_1)\times(x-x_2)\ldots\times(x-x_n)$ 这个 n 次多项式，然后通过模拟**长除法**，$O(n)$时间内可以得到 $\frac{(x-x_1)\times(x-x_2)\ldots\times(x-x_n)}{(x-x_i)}$，然后常系数直接 $O(n)$ 暴力算出来，就得到了 x_i 对应的多项式，最后把所有多项式加起来就得到了最终的系数。

```
const int maxn = 5007;
ll mod = 998244353;

ll qpow(ll a, ll b) {
    ll res = 1;
    while (b) {
        if (b & 1)
            res = res * a % mod;
        a = a * a % mod, b >>= 1;
    }
    return res;
}
ll a[maxn], b[maxn], c[maxn], temp[maxn];
ll x[maxn], y[maxn];
int n;
void mul(ll *f, int len, ll t) { //len为多项式的次数+1，函数让多项式f变成f*(x+t)
    for (int i = len; i > 0; --i)
        temp[i] = f[i], f[i] = f[i - 1];
    temp[0] = f[0], f[0] = 0;
    for (int i = 0; i ≤ len; ++i)
        f[i] = (f[i] + t * temp[i]) % mod;
}
void dev(ll *f, ll *r, ll t) { //f是被除多项式的系数，r保存f除以x+t的结果
    for (int i = 0; i ≤ n; ++i)
        temp[i] = f[i];
    for (int i = n; i > 0; --i) {
        r[i - 1] = temp[i];
        temp[i - 1] = (temp[i - 1] - t * temp[i]) % mod;
    }
    return;
}
void lglr() {
```

```
    memset(a, 0, sizeof a);
    b[1] = 1, b[0] = -x[1];
    for (int i = 2; i ≤ n; ++i) {
        mul(b, i, -x[i]);
    }//预处理(x-x1)*(x-x2)...*(x-xn)
    for (int i = 1; i ≤ n; ++i) {
        ll fz = 1;
        for (int j = 1; j ≤ n; ++j) {
            if (j == i)
                continue;
            fz = fz * (x[i] - x[j]) % mod;
        }
        fz = qpow(fz, mod - 2);
        fz = fz * y[i] % mod; //得到多项式系数
        dev(b, c, -x[i]);//得到多项式，保存在b数组
        for (int j = 0; j < n; ++j)
            a[j] = (a[j] + fz * c[j]) % mod;
    }
}
int main() {
    ll k;
    cin >> n >> k;
    for (int i = 1; i ≤ n; ++i)
        scanf("%lld%lld", &x[i], &y[i]);
    lglr();
    ll ans = 0;
    ll res = 1;
    for (int i = 0; i < n; ++i) {
        ans = (ans + res * a[i]) % mod;
        res = res * k % mod;
    }
    ans = (ans + mod) % mod;
    cout << ans << endl;
}

```

拉格朗日插值法 在ACM竞赛中的一些应用

0x12.牛顿插值

待更hhh

关于多项式的一点研究及其在ACM竞赛中的应用

0x20.多项式乘法

0x21.快速傅里叶变换（FFT）

> 如果还没有学FFT的话可以点开下面链接学习一下，上万字长文带你速通FFT
>
> - （子博客）链接：**【学习笔记】超简单的快速傅里叶变换（FFT）**包含全套证明，这里只说一些重点，具体的证明在链接里

0x21.1 FFT 简述

我们已知一个 $(n-1)$ 次多项式 $A(x) = \sum_{i=0}^{n-1} a_i x^i$ 进行了离散傅里叶变换后的点值 $\{d_i\}$，即

$d_k = \sum_{i=0}^{n-1} a_i \times \omega_n^{ik}$

这个过程为DFT，使用FFT在$O(nlogn)$的时间复杂度下完成这一操作。

现在试图还原系数数列 $\{a_i\}$。

结论：

$a_k = \frac{1}{n}\sum_{i=0}^{n-1} d_i\omega_n^{-ki}$

这个过程为IDFT，同样可以FFT在$O(nlogn)$的时间复杂度下完成这一操作。

如何使用FFT来计算 $B(x) = \sum_{i=0}^{n-1} b_i \times x^i B$

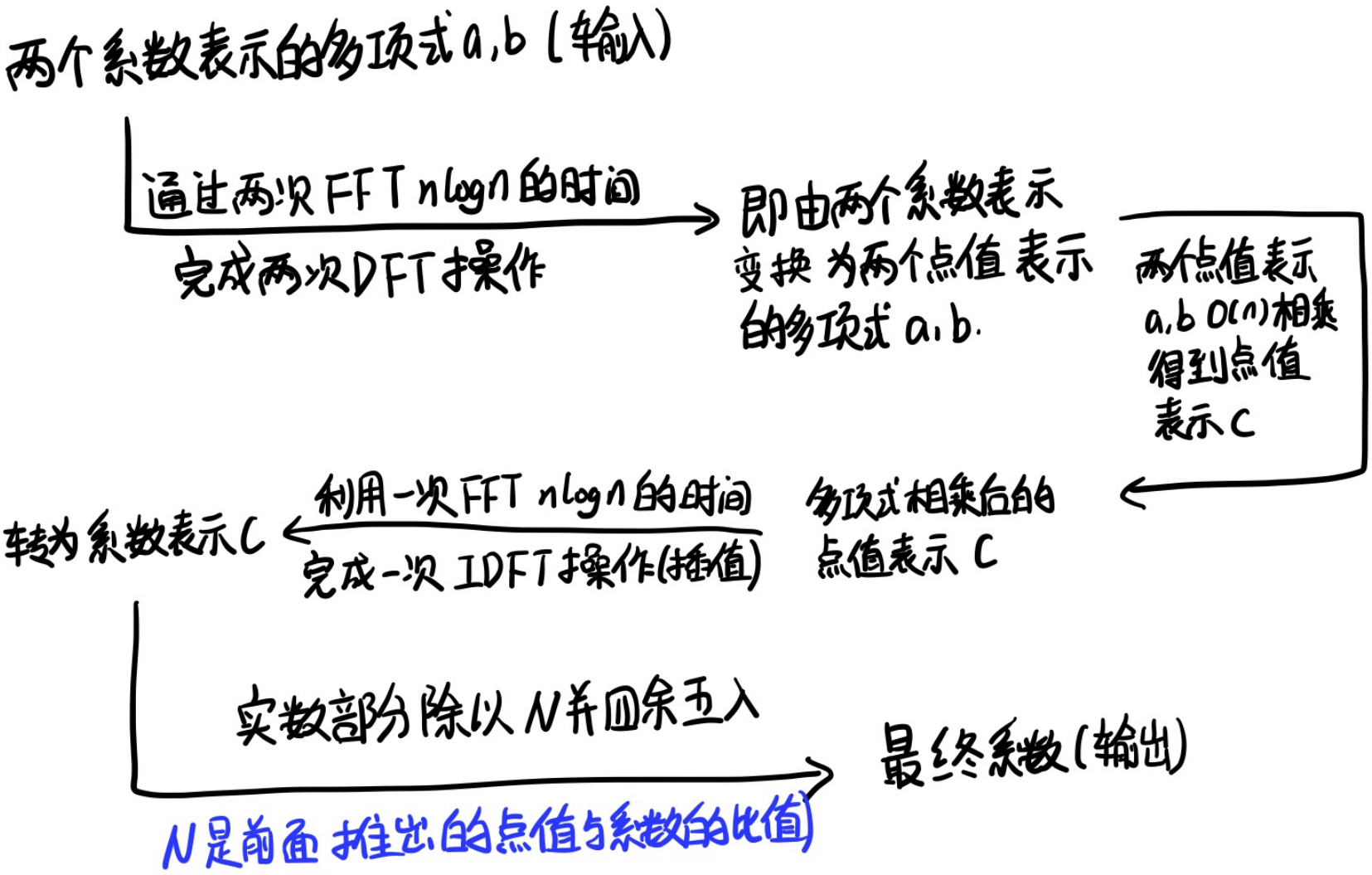
在 w_n^{-ki} ，其中 $0 \le k < n$ 处的点值。

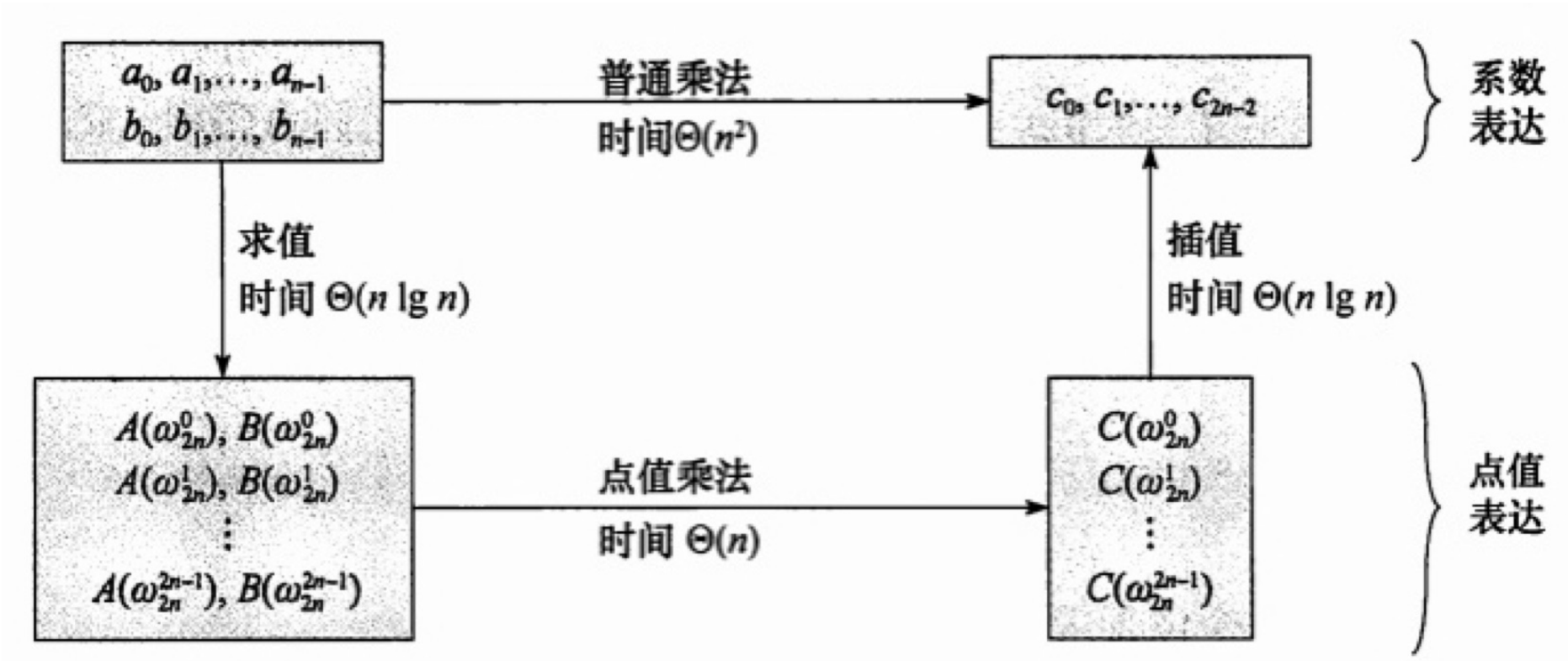
而 w_n^{-k} 可以看做 n 次本原单位根每次逆时针旋转本原单位根幅角的弧度，因此ω_n^{-k} 和 ω_n^k是一一对应的。具体的，$w_n^{-k} = w_n^{k+n}$ 。因此我们只需要使用 FFT 的方法，求出 $B(x)B$ 在 ω_n 各个幂次下的值，然后数组反过来，即令 $a_k = \frac{1}{n}\sum_{i=0}^{n} B(w_n^{n-k})$ 即可。

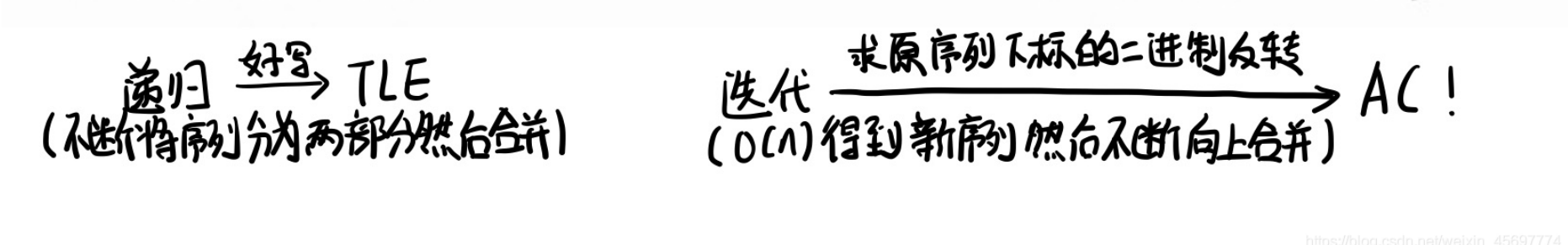
这一步快速计算插值的过程叫做快速傅里叶逆变换(`Inverse Fast Fourier Transform , IFFT`)。

至此，我们得到了一个时间复杂度为 $O(n \log n)$ 的多项式乘法计算方法。

算法流程：







https://blog.csdn.net/weixin_45697774

0x21.2. 递归版

我们把上述的思路实现用递归可以很形象地实现FFT函数。

这里要注意一下，尽管C++自带有复数库，但是建议手写一下，代码不长，手写常数小。

```
1 void FFT(int limit, Complex *a, int type) {
2     if (limit == 1) return ; //只有一个常数项
3     Complex a1[limit >> 1], a2[limit >> 1];
4     for (int i = 0; i ≤ limit; i += 2) //根据下标的奇偶性分类
5         a1[i >> 1] = a[i], a2[i >> 1] = a[i + 1];
```

```
    FFT(limit >> 1, a1, type);
    FFT(limit >> 1, a2, type);
    Complex Wn = Complex(cos(2.0 * Pi / limit) , type * sin(2.0 * Pi / limit)), w = Complex(1, 0);
    //Wn为单位根，w表示幂
    for (int i = 0; i < (limit >> 1); i++, w = w * Wn) //这里的w相当于公式中的k
        a[i] = a1[i] + w * a2[i],
               a[i + (limit >> 1)] = a1[i] - w * a2[i]; //利用单位根的性质，O(1)得到另一部分
}

int main() {
    int N = read(), M = read();
    for (int i = 0; i ≤ N; i++) a[i].x = read();
    for (int i = 0; i ≤ M; i++) b[i].x = read();
    int limit = 1; while (limit ≤ N + M) limit <<= 1;
    FFT(limit, a, 1);
    FFT(limit, b, 1);
    //后面的1表示要进行的变换是什么类型
    //1表示从系数变为点值
    //-1表示从点值变为系数
    //至于为什么这样是对的，可以参考一下c向量的推导过程，
    for (int i = 0; i ≤ limit; i++)
        a[i] = a[i] * b[i];
    FFT(limit, a, -1);
    for (int i = 0; i ≤ N + M; i++) printf("%d ", (int)(a[i].x / limit + 0.5)); //按照我们推倒的公式，这里还要除以n
    return 0;
}
```

但是这样写常数太大了，还需要一个很大的数组存，实测会T飞

0x21.3. 迭代版（蝴蝶变换）

所以我们引入一种迭代版，利用蝴蝶变换巧妙地实现FFT，使其真正成为$O(nlogn)$的高效算法。

P3803 【模板】多项式乘法（FFT）

题目描述

给定一个 n 次多项式 $F(x)$，和一个 m 次多项式 $G(x)$。

请求出 $F(x)$ 和 $G(x)$ 的卷积。

```
const double PI = acos(-1);
int n, m;
int res, ans[N];
int limit = 1;//补齐的2的整数幂N
int L;//二进制的位数
int R[N];//二进制翻转
struct Complex
{
    double x, y;
    Complex (double x = 0, double y = 0) : x(x), y(y) { }
}a[N], b[N];
//复数乘法：模长相乘，幅度相加
Complex operator * (Complex J, Complex Q) {return Complex(J.x * Q.x - J.y * Q.y, J.x * Q.y + J.y * Q.x);}
Complex operator - (Complex J, Complex Q) {return Complex(J.x - Q.x, J.y - Q.y);}
Complex operator + (Complex J, Complex Q) {return Complex(J.x + Q.x, J.y + Q.y);}

void FFT(Complex * A, int type)
```

```
{
    for(int i = 0; i < limit; ++ i)
        if(i < R[i])
            swap(A[i], A[R[i]]);
        //i小于R[i]时才交换，防止同一个元素交换两次，回到它原来的位置。

    //从底层往上合并
    for(int mid = 1; mid < limit; mid <<= 1) {
        //待合并区间长度的一半，最开始是两个长度为1的序列合并,mid = 1;
        Complex wn(cos(PI / mid), type * sin(PI / mid));//单位根w_n^1;

        for(int len = mid << 1, pos = 0; pos < limit; pos += len) {
            //len是区间的长度，pos是当前的位置,也就是合并到了哪一位
            Complex w(1, 0);//幂,一直乘，得到平方，三次方 ...

            for(int k = 0; k < mid; ++ k, w = w * wn) {
                //只扫左半部分，蝴蝶变换得到右半部分的答案,w 为 w_n^k
                Complex x = A[pos + k];//左半部分
                Complex y = w * A[pos + mid + k];//右半部分
                A[pos + k] = x + y;//左边加
                A[pos + mid + k] = x - y;//右边减
            }
        }
    }
    if(type == 1) return ;
    for(int i = 0; i ≤ limit; ++ i)
        a[i].x /= limit;
        //最后要除以limit也就是补成了2的整数幂的那个N，将点值转换为系数
        //（前面推过了点值与系数之间相除是N）
}
int main()
{
    n = read(), m = read();
    //读入多项式的每一项，保存在复数的实部
    for(int i = 0; i ≤ n; ++ i)
        a[i].x = read();
    for(int i = 0; i ≤ m; ++ i)
        b[i].x = read();
    while(limit ≤ n + m)
        limit <<= 1, L ++ ;
    //也可以写成：limit = 1 << int(log2(n + m) + 1);
    // 补成2的整次幂，也就是N
    for(int i = 0; i < limit; ++ i)
        R[i] = (R[i >> 1] >> 1) | ((i & 1) << (L - 1));
    FFT(a, 1);//FFT 把a的系数表示转化为点值表示
    FFT(b, 1);//FFT 把b的系数表示转化为点值表示
    //计算两个系数表示法的多项式相乘后的点值表示
    for(int i = 0; i ≤ limit; ++ i)
        a[i] = a[i] * b[i];
    //对应项相乘，O(n)得到点值表示的多项式的解C，利用逆变换完成插值得到答案C的点值表示
    FFT(a, -1);

    for(int i = 0; i ≤ n + m; ++ i)
        //这里的 x 和 y 是 double 的 hhh
        printf("%d ", (int)(a[i].x + 0.5));//注意要+0.5，否则精度会有问题
}

```

0x21.4. “三步变两步”优化

设P和 Q 是 实多项式，$F = P + Qi$，则 $F^2 = P^2 - Q^2 + 2PQi$，注意到我们要求的PQ正是F 虚部的一半。这样只需要两次FFT就可以求出结果。

```
void FFT(Complex * A, int type)//FFT板子
{
    for(int i = 0; i < limit; ++ i)
        if(i < R[i])
            swap(A[i], A[R[i]]);

    for(int mid = 1; mid < limit; mid <<= 1) {
        Complex wn(cos(PI / mid), type * sin(PI / mid));

        for(int len = mid << 1, pos = 0; pos < limit; pos += len) {
            Complex w(1, 0);

            for(int k = 0; k < mid; ++ k, w = w * wn) {
                Complex x = A[pos + k];
                Complex y = w * A[pos + mid + k];
                A[pos + k] = x + y;
                A[pos + mid + k] = x - y;
            }
        }
    }
    if(type == 1) return ;
    for(int i = 0; i ≤ limit; ++ i)
        a[i].x /= limit, a[i].y /= limit;
}

int main()
{
    n = read(), m = read();
    for(int i = 0; i ≤ n; ++ i)
        a[i].x = read();
    for(int i = 0; i ≤ m; ++ i)
        a[i].y = read();//把b(x)放到a(x)的虚部上
    while(limit ≤ n + m)
        limit <<= 1, L ++ ;
    for(int i = 0; i < limit; ++ i)
        R[i] = (R[i >> 1] >> 1) | ((i & 1) << (L - 1));
    FFT(a, 1);
    for(int i = 0; i ≤ limit; ++ i)
        a[i] = a[i] * a[i];//求出a(x)^2
    FFT(a, -1);

    for(int i = 0; i ≤ n + m; ++ i)
        printf("%d ", (int)(a[i].y / 2 + 0.5));
        //虚部取出来除2，注意要+0.5，否则精度会有问题,这里的x和y都是double

}

```

0x22.快速数论变换（NTT）

> - （子博客）链接：**【学习笔记】快速数论变换（NTT）（含全套证明）** 包含全套证明，这里只说一些重点，具体的证明和拓展应用在链接里哈

0x22.1NTT简述

快速数论变换（Number Theoretic Transform）

NTT，同样可以用来加速多项式乘法卷积，用于解决多项式乘法带模的情况，精度更高

前置知识

1.生成子群

子群：群$(S,\oplus)$，$(S',\oplus)$，满足$S'\subset S$，则$(S',\oplus)$是$(S,\oplus)$的子群

2.原根

原根g满足 $ord_n(g)=|Z_n^x|=\varphi(n)$，对于质数$P$，也就是说 $g^i \bmod P$，$0\le i\le P-1$ 的结果互不相同.

模n有原根的充要条件：$n=2,4,p^e,2\times p^e$

对于$g,p\in Z$，若$g^i \bmod p$ $(1\le i\le p-1)$的值互不相同，则称g为P的原根.

即 $i\in[1,P-2]$，$g^i \bmod p\neq g^{\varphi(p)} \bmod p$ $(g^{p-1} \bmod p)$

常见模数原根：998244353，100453589，469762049. 原根均为3.

NTT

FTT利用单位根ω的性质实现分治优化多项式乘法，原根也有这样的性质.

NTT即用原根代替ω的FTT，这样可以不用复数且实现取模运算.

对于质数$P=qn+1$，$(n=2^m)$，原根g满足$g^{qn}\equiv 1 \pmod P$

将$g_n=g^q \pmod P$看做ω_n的等阶，选择二者相似的性质

如 $g_n^n\equiv 1 \pmod P$，$g_n^{\frac{n}{2}}\equiv -1 \pmod P$

即将 $g^{\frac{p-1}{n}} \pmod p$看作 $\omega_n=e^{-\frac{2\pi i}{n}}$的等阶，即 $e^{-\frac{2\pi i}{n}}\equiv g^{\frac{p-1}{n}} \pmod p$

具体实现：当合并区间长度为 $len=2mid$时，

单位根 $w_n^1=\cos\frac{2\pi}{len}+i\sin\frac{2\pi}{len}=\cos\frac{\pi}{mid}+i\sin\frac{\pi}{mid}$.

原根：$g^{\frac{p-1}{len}}=g^{\frac{p-1}{2mid}}$

FTT $\longrightarrow$ NTT：

将单位复根→原根，复数对应在整数上，之后的一切的在 $mod\ p$ 环境下进行的

https://blog.csdn.net/weixin_45697774

总结：

利用单位根的FFT

$$DFT:\quad A(k)=\sum_{i=0}^{N-1}B(i)\,\omega_n^{ik}$$

$$IDFT:\quad B(k)=\frac{1}{N}\sum_{i=0}^{N-1}A(i)\,\omega_n^{-ki}$$

利用原根的NTT

$$DFT:\quad A(k)\equiv\sum_{i=0}^{N-1}B(i)\,a^{i\cdot k}\pmod M$$

$$IDFT:\quad B(k)\equiv\frac{1}{N}\sum_{i=0}^{N-1}A(i)\,a^{-ik}\pmod M$$

代码实现上相差无几.

注：这里的a^{-ik}指的是a^{ik}在模M意义下的逆元

模板题目大意：输入 n 次多项式 $f(x)$ 和 m 次多项式$g(x)$，求两个多项式的卷积

这里我们把多项式乘法封装成函数，更容易调用（NTT是多项式问题中最常用的变换）

```cpp
const int p = 998244353, G = 3, Gi = 332748118;//这里的Gi是G的除法逆元
const int N = 5000007;

const double PI = acos(-1);

int n, m;
int res, ans[N];
int limit = 1;//
int L;//二进制的位数
int RR[N];
ll a[N], b[N];

inline int read()
{
    register int x = 0, f = 1;
    register char ch = getchar();
    while(ch < '0' || ch > '9') {if(ch == '-')f = -1;ch = getchar();}
    while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

ll qpow(ll a, ll b)
{
    ll res = 1;
    while(b) {
        if(b & 1) res = res * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return res % p;
}

ll inv(ll x) {return qpow(x, p - 2);}

void NTT(ll *A, int type)
{
    for(int i = 0; i < limit; ++ i)
        if(i < RR[i])
            swap(A[i], A[RR[i]]);
    for(int mid = 1; mid < limit; mid <<= 1) {//原根代替单位根
```

```
        //ll wn = qpow(type == 1 ? G : Gi, (p - 1) / (mid << 1));
        ll wn = qpow(G, (p - 1) / (mid * 2));
        if(type == -1) wn = qpow(wn, p - 2);
        //如果超时了上面if这句话删掉，在下面的if(type == -1)里加上下面这个循环
        /*for (int i = 1; i < limit / 2; i ++)
        swap(A[i], A[limit - i]); */
        //逆变换则乘上逆元,因为我们算出来的公式中逆变换是(a^-ij)，也就是(a^ij)的逆元
        for(int len = mid << 1, pos = 0; pos < limit; pos += len) {
            ll w = 1;
            for(int k = 0; k < mid; ++ k, w = (w * wn) % p) {
                int x = A[pos + k], y = w * A[pos + mid + k] % p;
                A[pos + k] = (x + y) % p;
                A[pos + k + mid] = (x - y + p) % p;

            }
        }
    }

    if(type == -1) {
        ll limit_inv = inv(limit);//N的逆元（N是limit, 指的是2的整数幂）
        for(int i = 0; i < limit; ++ i)
            A[i] = (A[i] * limit_inv) % p;//NTT还是要除以n的，但是这里把除换成逆元了，inv就是n在模p意义下的逆元
    }
}//代码实现上和FFT相差无几
//多项式乘法
void poly_mul(ll *a, ll *b, int deg)
{
    for(limit = 1, L = 0; limit ≤ deg; limit <<= 1) L ++ ;
    for(int i = 0; i < limit; ++ i) {
        RR[i] = (RR[i >> 1] >> 1) | ((i & 1) << (L - 1));
    }
    NTT(a, 1);
    NTT(b, 1);
    for(int i = 0; i < limit; ++ i) a[i] = a[i] * b[i] % p;
    NTT(a, -1);
}

int main()
{
    n = read(), m = read();
    for(int i = 0; i ≤ n; ++ i) a[i] = (read() + p) % p;//取模好习惯
    for(int i = 0; i ≤ m; ++ i) b[i] = (read() + p) % p;
    poly_mul(a, b, n + m);
    for(int i = 0; i ≤ n + m; ++ i)
        printf("%d ", a[i]);
    return 0;
}
```

下面的所有内容待更（待学），预计2021年前更完hhh

0x23.快速沃尔什变换（FWT）

《小学生都能看懂的快速沃尔什变换从入门到升天教程》（FWT / FMT / FMI）（最最严谨清晰的证明！零基础也能得学会！）

0x24.快速莫比乌斯变换（FMT）

0x25.任意模数变换（MTT）

P4239 任意模数多项式乘法逆

0x30.多项式求逆

- （子博客）链接：**【学习笔记】超简单的多项式求逆（含全套证明）** 包含全套证明，这里只说一些重点，具体的证明和拓展应用在链接里哈

0x31. 多项式求逆简述

多项式求逆

定义：设 $f(x)=\sum_{k=0}^{n-1}a_kx^k$，求 $g(x)=\sum_{k=0}^{n-1}b_kx^k$

使得 $f(x)\cdot g(x)\equiv 1\pmod{x^n}$

即 $f(x)\cdot g(x)$ 的前 n 项中只有常数项为 1，其余项均为 0

称为求 $f(x)$ 模 x^n 的逆元，简称求 $f(x)$ 的逆元.

多项式求逆当且仅当其常数项可逆.即 $b_0=\frac{1}{a_0}$.

方法一.分治FFT

求多项式 $a(x)$ 的逆 $b(x)$，使 $a(x)\cdot b(x)\equiv 1\pmod{x^n}$

比较常数项系数易得 $b_0=\frac{1}{a_0}$ 且 $\sum_{i=0}^{k}b_i\cdot a_{k-i}=0$

我们想求的是 b_k，即 b_k 与 a 的关系

故我们将 $\sum_{i=0}^{k}b_i\cdot a_{k-i}=0$ 中提出 b_k 得

$$b_k=\sum_{i=0}^{k-1}b_i\left(-\frac{a_{k-i}}{a_0}\right)$$

即可使用FFT求解，时间复杂度 $O(n\log^2 n)$

下面介绍更优复杂度的做法：$O(n\log n)$ 的倍增法

方法二：倍增法

由于FFT使用的是分治+递推，考虑加速递推的老朋友倍增

很显然我们应该从局部的多项式逆元(从低次递推到高次)出发.

设 $h(x)=g(x)\pmod{x^{\lceil\frac{n}{2}\rceil}}$（名义求 $\bmod x^n$，先考虑 $\bmod x^{\lceil\frac{n}{2}\rceil}$ 意义下的逆元）

根据定义得：

$$f(x)\cdot h(x)\equiv 1\pmod{x^{\lceil\frac{n}{2}\rceil}} \quad (1)$$

$$f(x)\cdot g(x)\equiv 1\pmod{x^{\lceil\frac{n}{2}\rceil}} \quad (2)$$

两式相减

$$g(x)-h(x)\equiv 0\pmod{x^{\lceil\frac{n}{2}\rceil}}$$

两边平方

$$g^2(x)-2g(x)h(x)+h^2(x)\equiv 0\pmod{x^n}$$

（$f(x)\cdot h(x)\equiv 1\pmod{x^{\lceil\frac{n}{2}\rceil}}$）

（$f(x)\cdot g(x)\equiv 1\pmod{x^n}$）

两端乘上 $f(x)$

$$g(x)-2h(x)+f(x)\cdot h^2(x)\equiv 0\pmod{x^n}$$

整理可得

$$g(x)\equiv(2-f(x)\cdot h(x))\cdot h(x)\pmod{x^n}$$

故我们仅需递归算出模 $x^{\lceil\frac{n}{2}\rceil}$ 意义下的逆元，即为 $h(x)$，套用上面推导的公式，使用NTT/MTT加速即可.

https://blog.csdn.net/weixin_45697774

0x32. 多项式求逆模板

P4238 【模板】多项式乘法逆

题目描述

给定一个多项式 $F(x)$，请求出一个多项式 $G(x)$，满足 $F(x) * G(x) \equiv 1 (\mathrm{mod}\ x^n)$。系数对 998244353 取模。

输入格式

首先输入一个整数 n，表示输入多项式的项数。
接着输入 n 个整数，第 i 个整数 a_i 代表 $F(x)$ 次数为 $i-1$ 的项的系数。

输出格式

输出 n 个数字，第 i 个整数 b_i 代表 $G(x)$ 次数为 $i-1$ 的项的系数。保证有解。

https://blog.csdn.net/weixin_45697774

直接按照上述的思路实现一下模板即可。注意要中间 $h(x)$ 要取的是模 $x^{\lceil \frac{n}{2} \rceil}$ 意义下的 $g(x)$，所以我们应该把后面大于 deg 的全部置为 0（这里的 deg 实际上就是我们证明的时候用到的 n），我们可以直接使用 $fill$ 函数实现这一操作。

```
const int N = 5000007;
const int p = 998244353, g = 3, gg = 3, ig = 332738118;
int mod = p;
int n, m;
int limit = 1;
int L;
int R[N];
ll A[N], B[N];

template <typename T>void read(T &x)
{
    x = 0;
    register int f = 1;
    register char ch = getchar();
    while(ch < '0' || ch > '9') {if(ch == '-')f = -1;ch = getchar();}
    while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0';ch = getchar();}
    x *= f;
}

ll qpow(ll a, ll b)
{
    ll res = 1;
    while(b) {
        if(b & 1) res = res * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return res % p;
}

ll inv(ll x) {return qpow(x, p - 2);}

void NTT(ll *A, int type)
{
    for(int i = 0; i < limit; ++ i) {
        if(i < R[i])
            swap(A[i], A[R[i]]);
    }
```

```
    for(int mid = 1; mid < limit; mid <<= 1) {
        ll wn = qpow(g, (p - 1) / (mid * 2));
        if(type == -1) wn = qpow(wn, p - 2);
        //如果超时了上面if这句话删掉，在下面的if(type == -1)里加上下面这个循环
        /*for (int i = 1; i < limit / 2; i ++)
        swap(A[i], A[limit - i]); */
        for(int len = mid << 1, pos = 0; pos < limit; pos += len) {
            ll w = 1;
            for(int k = 0; k < mid; ++ k, w = (w * wn) % p) {
                int x = A[pos + k] % p, y = w * A[pos + mid + k] % p;
                A[pos + k] = (x + y) % p;
                A[pos + k + mid] = (x - y + p) % p;
            }
        }
    }
    if(type == 1) return ;
    ll inv_limit = inv(limit);
    for(int i = 0; i < limit; ++ i)
        A[i] = (A[i] * inv_limit) % p;
}

//。C[N] == f[N] % x ^ deg;
//。B[N] == h[N] == g[N] % x ^ [deg/2]
//。A[N] == f[N];
ll C[N];

void get_inv(ll *A, ll *B, int deg)//deg = 多项式的度
{
    if(deg == 1) {
        B[0] = inv(A[0]);//A[0]的逆即常数项
        return ;
    }

    get_inv(A, B, (deg + 1) >> 1);//递归分治
    for(limit = 1; limit ≤ (deg << 1); limit <<= 1);
    for(int i = 0; i < limit; ++ i) {
        R[i] = (R[i >> 1] >> 1) | ((i & 1) ? (limit >> 1) : 0);
        C[i] = (i < deg ? A[i] : 0);//只算到[n/2]，后面的全部为0;
    }
    NTT(C, 1), NTT(B, 1);
    for(int i = 0; i < limit; ++ i) {
        B[i] = (2ll - C[i] * B[i] % p + p) % p * B[i] % p;
    }
    NTT(B, -1);
    fill(B + deg, B + limit, 0);//非常重要，因为是在模 x^deg 意义下 ，所以大于deg的置0
}

int main()
{
    read(n);
    for(int i = 0; i < n; ++ i) read(A[i]);
    get_inv(A, B, n);
    for(int i = 0; i < n; ++ i) {
        printf("%lld ", B[i]);
    }
    return 0;
}
```

0x40.多项式开方

- （子博客）链接： **【学习笔记】多项式开方** 包含全套证明，这里只说一些重点，具体的证明和拓展应用在链接里哈

0x41. 多项式开方简述

多项式开方

给定 $n-1$ 次多项式 $g(x)$，求在 $\bmod x^n$ 意义下的 $f(x)$，使得 $f^2(x) \equiv g(x) \pmod{x^n}$ （$f(x) \equiv \sqrt{g(x)} \pmod{x^n}$）

若有多解，取零次项系数小的作为答案

倍增(分治)法

假设现在已求出 $g(x)$ 在模 $x^{\lceil \frac{n}{2} \rceil}$ 意义下的平方根 $f_0(x)$

则：

$$
\begin{aligned}
f_0^2(x) &\equiv g(x) && \pmod{x^{\lceil \frac{n}{2} \rceil}} \\
f_0^2(x) - g(x) &\equiv 0 && \pmod{x^{\lceil \frac{n}{2} \rceil}} \\
(f_0^2(x) - g(x))^2 &\equiv 0 && \pmod{x^n} \\
(f_0^2(x) + g(x))^2 &\equiv 4 f_0^2(x) \cdot g(x) && \pmod{x^n} \\
\left(\frac{f_0^2(x) + g(x)}{2 f_0(x)}\right)^2 &\equiv g(x) && \pmod{x^n} \\
\frac{f_0^2(x) + g(x)}{2 f_0(x)} &\equiv f(x) && \pmod{x^n} \\
2^{-1} f_0(x) + 2^{-1} f_0^{-1}(x) g(x) &\equiv f(x) && \pmod{x^n}
\end{aligned}
$$

时间复杂度 $T(n) = T(\frac{n}{2}) + O(n \log n) = O(n \log^2 n)$ （每次需要 $O(\log n)$ 求逆）

https://blog.csdn.net/weixin_45697774

0x42. 多项式开方(根)模板

题目描述

给定一个$n-1$次多项式$A(x)$，求一个在$\bmod x^n$意义下的多项式$B(x)$，使得$B^2(x) \equiv A(x) \pmod{x^n}$。若有多解，请取零次项系数较小的作为答案。

多项式的系数在$\bmod 998244353$的意义下进行运算。

还有一种常数较小的写法就是在倍增维护 $f(x)$ 的时候同时维护 $f^{-1}(x)$ 而不是每次都求逆。

> 当 $[x^0] g(x) \neq 1$ 时，可能需要使用二次剩余来计算 $[x^0] f(x)$ 。

```
const int N = 5000007;
const int p = 998244353, gg = 3, ig = 332738118;
const int mod = 998244353;
int limit = 1;
int L;
int R[N];
ll f[N], g[N], h[N];
```

```
ll A[N], B[N], C[N];
//部分模板代码省略
template <typename T>void read(T &x);//快读
ll qpow(ll a, ll b);//快速幂
ll inv(ll x) {return qpow(x, mod - 2);}//逆元
int inv2 = qpow(2, mod - 2);
void NTT(ll *A, int type);//快速数论变换
void get_inv(ll *A, ll *B, int deg);//多项式求逆
void poly_sqrt(ll *f, ll *h, int deg) {//polynomial 多项式
    if (deg == 1) {
        h[0] = 1;
        return;
    }

    poly_sqrt(f, h, deg + 1 >> 1);

    limit = 1;
    while (limit < deg << 1) {
        limit <<= 1;
    }
    fill(g, g + limit, 0);
    get_inv(h, g, deg);
    copy(f, f + deg, t);
    fill(t + deg, t + limit, 0);
    NTT(t, 1);
    NTT(g, 1);
    NTT(h, 1);

    for (int i = 0; i < limit; i++) {
        h[i] = 1LL * inv2 * (1LL * h[i] % mod + 1LL * g[i] * t[i] % mod) % mod;
    }
    NTT(h, -1);
    fill(h + deg, h + limit, 0);
}
int n, m;
ll a[N], b[N];

//输入g输出f中间多项式h
//h是g的一半的平方根
int main()
{
    read(n);
    for(int i = 0; i < n; ++ i) read(a[i]);

    poly_sqrt(a, b, n);

    for(int i = 0; i < n; ++ i)
        printf("%lld ", b[i]);
    return 0;
}


```

0x50.多项式除法 / 取模

> - （子博客）链接： **【学习笔记】多项式除法（含完整证明）** 包含全套证明，这里只说一些重点，具体的证明和拓展应用在链接里哈

0x51.多项式除法简述

> 题目大意：给定一个n次多项式$F(x)$，m次多项式$G(x)$，求一个$n-m$次多项式$Q(x)$，和一个小于m次的多项式$R(x)$。

满足$F(x)=G(x)Q(x)+R(x)$

即求$F(x)$除以$G(x)$得到的商$Q(x)$以及余数$R(x)$。

定义$Fr(x)$为多项式$F(x)$系数翻转后的多项式，即

$F(x)=\sum_{i=0}^{n}a_ix^i$，$G(x)=\sum_{i=0}^{n}a_{n-i}x^i$

翻转的实现可以使用公式$Fr(x)=x^nF(x^{-1})\rightarrow F(x)=x^nFr(x^{-1})$，也可以直接循环来交换系数来实现

最终推导得到：$Fr(x)\equiv Q(x)Gr(x)\ (mod\ x^{n-m+1})$

即：

$Qr(x)\equiv Fr(x)Gr^{-1}(x)\ (mod\ x^{n-m+1})$

$R(x)=F(x)-G(x)Q(x)$

简单实现一下即可。

时间复杂度：$O(nlogn)$

0x52. 多项式除法模板

```cpp
const int N = 4000007;
const int p = 998244353, gg = 3, ig = 332738118;
const int mod = 998244353;
int limit = 1;
int L;
int RR[N];
ll F[N], G[N], H[N], Q[N], R[N];
ll A[N], B[N], C[N];
ll gg_inv;

template <typename T>void read(T &x)
{
    x = 0;
    register int f = 1;
    register char ch = getchar();
    while(ch < '0' || ch > '9') {if(ch == '-')f = -1;ch = getchar();}
    while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0';ch = getchar();}
    x *= f;
}

ll qpow(ll a, ll b)
{
    ll res = 1;
    while(b) {
        if(b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res % p;
}

ll inv(ll x) {return qpow(x, mod - 2);}
```

```

void NTT(ll *A, int type)
{
    for(int i = 0; i < limit; ++ i)
        if(i < RR[i])
            swap(A[i], A[RR[i]]);
    for(int mid = 1; mid < limit; mid <<= 1) {
        ll wn = qpow(gg, (mod - 1) / (2 * mid));
        if(type == -1) wn = qpow(wn, mod - 2);
        for(int len = mid << 1, pos = 0; pos < limit; pos += len) {
            ll w = 1;
            for(int k = 0; k < mid; ++ k, w = (w * wn) % mod){
                int x = A[pos + k] % mod;
                int y = w * A[pos + k + mid] % mod;
                A[pos + k] = (x + y) % mod;
                A[pos + k + mid] = (x - y + mod) % mod;
            }
        }
    }
    if(type == -1) {
        ll limit_inv = inv(limit);
        for(int i = 0; i < limit; ++ i) {
            A[i] = (A[i] * limit_inv) % mod;
        }
    }
}

//多项式求逆
void get_inv(ll *A, ll *B, int deg)
{
    if(deg == 1) {
        B[0] = inv(A[0]);
        return ;
    }

    get_inv(A, B, (deg + 1) >> 1);

    for(limit = 1, L = 0; limit ≤ (deg << 1) * 2; limit <<= 1) L ++ ;
    for(int i = 0; i < limit; ++ i) {
        RR[i] = (RR[i >> 1] >> 1) | ((i & 1) << (L - 1));
        C[i] = (i < deg ? A[i] : 0);
    }
    NTT(C, 1), NTT(B, 1);
    for(int i = 0; i ≤ limit; ++ i)
        B[i] = (2ll - C[i] * B[i] % mod + mod) % mod * B[i] % mod;
    NTT(B, -1);
    fill(B + deg, B + limit, 0);
}

void poly_mul(ll *a, ll *b, int n, int m)
{
    for(limit = 1, L = 0; limit ≤ (n + m) * 2; limit <<= 1) L ++ ;
    for(int i = 0; i < limit; ++ i) {
        RR[i] = (RR[i >> 1] >> 1) | ((i & 1) << (L - 1));
    }
    NTT(a, 1);
    NTT(b, 1);
    for(int i = 0; i < limit; ++ i ) {
        a[i] = a[i] * b[i] % mod;
    }
```

```
    NTT(a, -1);
}

ll t[N], Fr[N], Gr[N], Qr[N], Gr_inv[N];
int n, m;

int main()
{
    //gg_inv = inv(gg);
    read(n);
    read(m);
    //n ++ , m ++ ;
    for(int i = 0; i ≤ n; ++ i) read(F[i]), Fr[n - i] = F[i];//读入F，求翻转的Fr数组（可以直接手动翻转hhh）
    for(int i = 0; i ≤ m; ++ i) read(G[i]), Gr[m - i] = G[i];//读入G，求翻转的Gr数组
    for(int i = n - m + 2; i ≤ m; ++ i) Gr[i] = 0;//Gr数组应该只有n - m项
    get_inv(Gr, Gr_inv, n - m + 1);//求逆

    poly_mul(Fr, Gr_inv, n, n - m);//乘起来得到Qr

    for(int i = 0; i ≤ n - m; ++ i) Q[i] = Fr[n - m - i];//求Qr数组的翻转既是要求的Q数组（商）

    for(int i = 0; i ≤ n - m; ++ i) printf("%lld ", Q[i]);

    puts("");

    poly_mul(G, Q, m, n - m);
    //R(x) = F(x) - G(x)Q(x);小于m项
    for(int i = 0; i < m; ++ i) R[i] = (F[i] - G[i] + mod) % mod;

    for(int i = 0; i < m; ++ i)
        printf("%lld ", R[i]);

    puts("");

    return 0;
}
```

vector 版本还没有调出来呜呜呜

0x60.多项式牛顿迭代

> - （子博客）链接： **【学习笔记】多项式牛顿迭代（含泰勒展开式、牛顿迭代全套证明）** 包含全套证明，这里只说一些重点，具体的证明和拓展应用在链接里哈

0x61. 多项式牛顿迭代及其应用

多项式牛顿迭代应用

$$f(x)\equiv f(x_0)-\frac{g(f(x_0))}{g'(f(x_0))},\ f(x_0)\equiv f(x)\pmod{x^{\lceil\frac{n}{2}\rceil}}$$

多项式求逆（$g(x)\cdot f(x)\equiv 1\pmod{x^n}$）

设给定函数$g(x)$

有方程：$h(f(x))=\frac{1}{f(x)}-g(x)\equiv 0\pmod{x^n}$

Newton's Method:

$$f(x)\equiv f(x_0)-\frac{\frac{1}{f(x_0)}-g(x)}{-\frac{1}{f^2(x_0)}}\pmod{x^n}$$

$$f(x)\equiv 2f(x_0)-f^2(x_0)g(x)\pmod{x^n}$$

多项式开方（$g(x)\equiv f^2(x)\pmod{x^n}$）

设给定函数$g(x)$

有方程：$h(f(x))=f^2(x)-g(x)\equiv 0\pmod{x^n}$

Newton's Method:

$$f(x)\equiv f(x_0)-\frac{f^2(x_0)-g(x)}{2f(x_0)}\pmod{x^n}$$

$$f(x)\equiv\frac{f^2(x_0)+g(x)}{2f(x_0)}\pmod{x^n}$$

多项式 exp（$g(x)=e^{f(x)}\pmod{x^n}$）

设给定函数$g(x)$

有方程：$h(f(x))=\ln f(x)-g(x)\equiv 0\pmod{x^n}$

Newton's Method:

$$f(x)\equiv f(x_0)-\frac{\ln f(x_0)-g(x)}{\frac{1}{f(x_0)}}$$

$$\equiv f(x_0)(1-\ln f(x_0)+g(x))$$

$$T(n)=T(\tfrac{n}{2})+O(n\log n)=O(n\log n)$$

https://blog.csdn.net/weixin_45697774

0x70.多项式多点求值 / 快速插值

0x80.多项式三角函数

【学习笔记】超简单的多项式三角函数（含全套证明）

$$g(x)\equiv\cos f(x)\equiv\frac{e^{i\cdot f(x)}+e^{-i\cdot f(x)}}{2}(\mathrm{mod}\ \ x^n)\tag{7}$$

$$g(x)\equiv \sin f(x)\equiv \frac{e^{i\cdot f(x)}-e^{-i\cdot f(x)}}{2i}(\mathrm{mod}\ \ x^n) \tag{8}$$

其中：

$i^2\equiv -1\equiv 998244352(\mathrm{mod}998244353)$

较小解为：

$i=86583718$

0x90.多项式反三角函数

【学习笔记】超简单的多项式反三角函数（含全套证明）

反正弦函数

$$g(x)\equiv \int \frac{f'(x)}{\sqrt{1-f^2(x)}}dx\quad (\mathrm{mod}\ \ x^n) \tag{9}$$

反正切函数

$$g(x)\equiv \int \frac{f'(x)}{1+f^2(x)}dx\quad (\mathrm{mod}\ \ x^n) \tag{10}$$

0x100.常系数齐次线性递推（多项式优化）

0x110.快速差分/前缀和

0x120.多项式对数（多项式求导 + 积分）

> - （子博客）链接： **【学习笔记】多项式对数（多项式求导 + 积分）** 包含全套证明，这里只说一些重点，具体的证明和拓展应用在链接里哈

0x121. 多项式对数简述（含常见函数求导表）

其实非常简单，就是直接求导再积分即可。

多项式对数

给定一个 n 次多项式 $f(x)$，求一个 $g(x)$ 满足：$g(x) \equiv \ln f(x) \pmod{x^n}$

无法直接计算 $\ln f(x)$，考虑求导再积分

$$(\ln f(x))' = \frac{f'(x)}{f(x)}$$

$$\ln f(x) = \int \frac{f'(x)}{f(x)} dx$$

$$g(x) = \int \frac{f'(x)}{f(x)} dx$$

然后就是大一学过大二忘的一干二净的高数了：

① $(u \pm v)' = u' \pm v'$

② $(Cu)' = Cu'$

③ $(uv)' = u'v + uv'$

④ $\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$

⑤ 若 $f(x) = a[b(x)]$
则 $f'(x) = a'(b) \cdot b'(x)$

$f(x)$	$f'(x)$
C	0
x^a	$a \cdot x^{a-1}$
a^x	$a^x \cdot \ln a$
e^x	e^x
$\log_a x$	$\frac{1}{x \cdot \ln a}$
$\ln x$	$\frac{1}{x}$
$\ln \lvert x \rvert$	
$\sin x$	$\cos x$
$\cos x$	$-\sin x$
$\tan x$	$\sec^2 x = \frac{1}{\cos^2 x} = 1 + \tan^2 x$
$\cot x = \frac{1}{\tan x}$	$-\csc^2 x = -\frac{1}{\sin^2 x} = -1 - \cot^2 x$
$\sec x = \frac{1}{\cos x}$	$\sec x \cdot \tan x = \frac{\sin x}{\cos^2 x}$
$\csc x = \frac{1}{\sin x}$	$-\csc x \cdot \cot x = -\frac{\cos x}{\sin^2 x}$
$\arcsin x$	$\frac{1}{\sqrt{1-x^2}}$
$\arccos x$	$-\frac{1}{\sqrt{1-x^2}}$
$\arctan x$	$\frac{1}{1+x^2}$
$\operatorname{arccot} x$	$-\frac{1}{1+x^2}$

小技巧：

$$a(x)^{b(x)} = e^{b(x) \cdot \ln a(x)}$$

$$\Downarrow$$

$$[a(x)^{b(x)}]' = [e^{b(x) \cdot \ln a(x)}]'$$

https://blog.csdn.net/weixin_45697774

0x122. 多项式对数函数（多项式 ln）模板

题目描述

给出 $n-1$ 次多项式 $A(x)$，求一个 $\bmod x^n$ 下的多项式 $B(x)$，满足 $B(x) \equiv \ln A(x)$.

在 $\bmod 998244353$ 下进行，且 $a_i \in [0, 998244353) \cap \mathbb{Z}$

直接简单实现一下就行了。

我们输入多项式 f，先求f的导数 A，f 的逆元 B，然后把他们乘起来（NTT），最后求积分 g 既是答案，输出即可。

至于求导和积分，因为我们的多项式$f(x)=a_0+a_1\times x+a_2\times x^2\dots a_n\times x^{n-1}$

$(x^a)'=ax^{a-1}$，$\int x^a dx=\frac{1}{a+1}x^{a+1}$

```cpp
const int N = 5000007;
const int p = 998244353, gg = 3, ig = 332738118;
const int mod = 998244353;
int limit = 1;
int L;
int RR[N];
ll f[N], g[N];
ll A[N], B[N], C[N];
//省略部分上面已有的代码
template <typename T>void read(T &x);//快读
ll qpow(ll a, ll b);//快速幂
ll inv(ll x) {return qpow(x, mod - 2);}//求乘法逆元
void NTT(ll *A, int type);//快速数论变换
void get_inv(ll *A, ll *B, int deg);//多项式求逆
//多项式求导
void get_dev(ll *A, ll *B, int n)
{
    for(int i = 1; i < n; ++ i)
        B[i - 1] = i * A[i] % mod;
    B[n - 1] = 0;
}
//多项式求积分
void get_indev(ll *A, ll *B, int n)
{
    for(int i = 1; i < n; ++ i)
        B[i] = A[i - 1] * qpow(i, mod - 2) % mod;
    B[0] = 0;
}

//多项式乘法
void poly_mul(ll *a, ll *b, int deg)
{
    for(limit = 1, L = 0; limit ≤ deg; limit <<= 1) L ++ ;
    for(int i = 0; i < limit; ++ i) {
        RR[i] = (RR[i >> 1] >> 1) | ((i & 1) << (L - 1));
    }
    NTT(a, 1);
    NTT(b, 1);
    for(int i = 0; i < limit; ++ i) a[i] = a[i] * b[i] % mod;
    NTT(a, -1);
}
//多项式对数
void get_ln(ll *f, ll *g, int n)
{
    get_dev(f, A, n);
    get_inv(f, B, n);
    poly_mul(A, B, n);
    get_indev(A, g, n);
}

int n, m;

int main()
{
    read(n);
```

```
    for(int i = 0; i < n; ++ i) read(f[i]);
    for(limit = 1; limit ≤ n; limit <<= 1);
    get_ln(f, g, limit);

    for(int i = 0; i < n; ++ i)
        printf("%d ", g[i]);
    puts("");
    return 0;
}

```

0x130.多项式指数（大毒瘤）

> - （子博客）链接： **【学习笔记】多项式求指（含泰勒展开式、牛顿迭代完成证明）** 包含全套证明，这里只说一些重点，具体的证明和拓展应用在链接里哈

由牛顿迭代得到的指数公式：

$f(x) \equiv f(x_0)(1 - ln(f(x_0)) + g(x))\ mod(\ x^n)$

代码见下面的全家桶

0x140.多项式快速幂

我们知道指数有一个性质：

$lnx^n = n\ lnx$

我们要求的是$f(x)^k$

可以很自然地构造出：

$f(x)^k = e^{lnf(x)^k} = e^{k\ lnf(x)}$

所以我们只需要求$f(x)$的 对数ln 乘上k然后求 指数 就行了QwQ

代码见下面的全家桶

0x150.多项式全家桶！！！（code）

“优 雅”

```
//#pragma GCC optimize(2)
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

const int N = 3000007;
const int p = 998244353, gg = 3, ig = 332738118, img = 86583718;
const int mod = 998244353;

template <typename T>void read(T &x)
{
    x = 0;
    register int f = 1;
    register char ch = getchar();
    while(ch < '0' || ch > '9') {if(ch == '-')f = -1;ch = getchar();}
    while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0';ch = getchar();}
    x *= f;
```

```
}

int qpow(int a, int b)
{
    int res = 1;
    while(b) {
        if(b & 1) res = 1ll * res * a % mod;
        a = 1ll * a * a % mod;
        b >>= 1;
    }
    return res;
}

namespace Poly
{
    #define mul(x, y) (1ll * x * y ≥ mod ? 1ll * x * y % mod : 1ll * x * y)
    #define minus(x, y) (1ll * x - y < 0 ? 1ll * x - y + mod : 1ll * x - y)
    #define plus(x, y) (1ll * x + y ≥ mod ? 1ll * x + y - mod : 1ll * x + y)
    #define ck(x) (x ≥ mod ? x - mod : x)//取模运算太慢了

    typedef vector<int> poly;
    const int G = 3;//根据具体的模数而定，原根可不一定不一样！！！
    //一般模数的原根为 2 3 5 7 10 6
    const int inv_G = qpow(G, mod - 2);
    int RR[N], deer[2][19][N], inv[N];

    void init(const int t) {//预处理出来NTT里需要的w和wn，砍掉了一个log的时间
        for(int p = 1; p ≤ t; ++ p) {
            int buf1 = qpow(G, (mod - 1) / (1 << p));
            int buf0 = qpow(inv_G, (mod - 1) / (1 << p));
            deer[0][p][0] = deer[1][p][0] = 1;
            for(int i = 1; i < (1 << p); ++ i) {
                deer[0][p][i] = 1ll * deer[0][p][i - 1] * buf0 % mod;//逆
                deer[1][p][i] = 1ll * deer[1][p][i - 1] * buf1 % mod;
            }
        }
        inv[1] = 1;
        for(int i = 2; i ≤ (1 << t); ++ i)
            inv[i] = 1ll * inv[mod % i] * (mod - mod / i) % mod;
    }

    int NTT_init(int n) {//快速数论变换预处理
        int limit = 1, L = 0;
        while(limit ≤ n) limit <<= 1, L ++ ;
        for(int i = 0; i < limit; ++ i)
            RR[i] = (RR[i >> 1] >> 1) | ((i & 1) << (L - 1));
        return limit;
    }

    void NTT(poly &A, int type, int limit) {//快速数论变换
        A.resize(limit);
        for(int i = 0; i < limit; ++ i)
            if(i < RR[i])
                swap(A[i], A[RR[i]]);
        for(int mid = 2, j = 1; mid ≤ limit; mid <<= 1, ++ j) {
            int len = mid >> 1;
            for(int pos = 0; pos < limit; pos += mid) {
                int *wn = deer[type][j];
                for(int i = pos; i < pos + len; ++ i, ++ wn) {
                    int tmp = 1ll * (*wn) * A[i + len] % mod;
```

```
78                  A[i + len] = ck(A[i] - tmp + mod);
79                  A[i] = ck(A[i] + tmp);
80              }
81          }
82      }
83      if(type == 0) {
84          for(int i = 0; i < limit; ++ i)
85              A[i] = 1ll * A[i] * inv[limit] % mod;
86      }
87  }
88
89  poly poly_mul(poly A, poly B) {//多项式乘法
90      int deg = A.size() + B.size() - 1;
91      int limit = NTT_init(deg);
92      poly C(limit);
93      NTT(A, 1, limit);
94      NTT(B, 1, limit);
95      for(int i = 0; i < limit; ++ i)
96          C[i] = 1ll * A[i] * B[i] % mod;
97      NTT(C, 0, limit);
98      C.resize(deg);
99      return C;
100 }
101
102 poly poly_inv(poly &f, int deg) {//多项式求逆
103     if(deg == 1)
104         return poly(1, qpow(f[0], mod - 2));
105
106     poly A(f.begin(), f.begin() + deg);
107     poly B = poly_inv(f, (deg + 1) >> 1);
108     int limit = NTT_init(deg << 1);
109     NTT(A, 1, limit), NTT(B, 1, limit);
110     for(int i = 0; i < limit; ++ i)
111         A[i] = B[i] * (2 - 1ll * A[i] * B[i] % mod + mod) % mod;
112     NTT(A, 0, limit);
113     A.resize(deg);
114     return A;
115 }
116
117 poly poly_dev(poly f) {//多项式求导
118     int n = f.size();
119     for(int i = 1; i < n; ++ i) f[i - 1] = 1ll * f[i] * i % mod;
120     return f.resize(n - 1), f;//f[0] = 0，这里直接扔了,从1开始
121 }
122
123 poly poly_idev(poly f) {//多项式求积分
124     int n = f.size();
125     for(int i = n - 1; i ; -- i) f[i] = 1ll * f[i - 1] * inv[i] % mod;
126     return f[0] = 0, f;
127 }
128
129 poly poly_ln(poly f, int deg) {//多项式求对数
130     poly A = poly_idev(poly_mul(poly_dev(f), poly_inv(f, deg)));
131     return A.resize(deg), A;
132 }
133
134 poly poly_exp(poly &f, int deg) {//多项式求指数
135     if(deg == 1)
136         return poly(1, 1);
137
```

```
        poly B = poly_exp(f, (deg + 1) >> 1);
        B.resize(deg);
        poly lnB = poly_ln(B, deg);
        for(int i = 0; i < deg; ++ i)
            lnB[i] = ck(f[i] - lnB[i] + mod);

        int limit = NTT_init(deg << 1);//n -> n^2
        NTT(B, 1, limit), NTT(lnB, 1, limit);
        for(int i = 0; i < limit; ++ i)
            B[i] = 1ll * B[i] * (1 + lnB[i]) % mod;
        NTT(B, 0, limit);
        B.resize(deg);
        return B;
    }

    poly poly_sqrt(poly &f, int deg) {//多项式开方
        if(deg == 1) return poly(1, 1);
        poly A(f.begin(), f.begin() + deg);
        poly B = poly_sqrt(f, (deg + 1) >> 1);
        poly IB = poly_inv(B, deg);
        int limit = NTT_init(deg << 1);
        NTT(A, 1, limit), NTT(IB, 1, limit);
        for(int i = 0; i < limit; ++ i)
            A[i] = 1ll * A[i] * IB[i] % mod;
        NTT(A, 0, limit);
        for(int i =0; i < deg; ++ i)
            A[i] = 1ll * (A[i] + B[i]) * inv[2] % mod;
        A.resize(deg);
        return A;
    }

    poly poly_pow(poly f, int k) {//多项式快速幂
        f = poly_ln(f, f.size());
        for(auto &x : f) x = 1ll * x * k % mod;
        return poly_exp(f, f.size());
    }

    poly poly_cos(poly f, int deg) {//多项式三角函数（cos）
        poly A(f.begin(), f.begin() + deg);
        poly B(deg), C(deg);
        for(int i = 0; i < deg; ++ i)
            A[i] = 1ll * A[i] * img % mod;

        B = poly_exp(A, deg);
        C = poly_inv(B, deg);
        int inv2 = qpow(2, mod - 2);
        for(int i = 0; i < deg; ++ i)
            A[i] = 1ll * (1ll * B[i] + C[i]) % mod * inv2 % mod;
        return A;
    }

    poly poly_sin(poly f, int deg) {//多项式三角函数（sin）
        poly A(f.begin(), f.begin() + deg);
        poly B(deg), C(deg);
        for(int i = 0; i < deg; ++ i)
            A[i] = 1ll * A[i] * img % mod;

        B = poly_exp(A, deg);
        C = poly_inv(B, deg);
        int inv2i = qpow(img << 1, mod - 2);
```

```
        for(int i = 0; i < deg; ++ i)
            A[i] = 1ll * (1ll * B[i] - C[i] + mod) % mod * inv2i % mod;
        return A;
    }

    poly poly_arcsin(poly f, int deg) {
        poly A(f.size()), B(f.size()), C(f.size());
        A = poly_dev(f);
        B = poly_mul(f, f);
        for(int i = 0; i < deg; ++ i)
            B[i] = minus(mod, B[i]);
        B[0] = plus(B[0], 1);
        C = poly_sqrt(B, deg);
        C = poly_inv(C, deg);
        C = poly_mul(A, C);
        C = poly_idev(C);
        return C;
    }

    poly poly_arctan(poly f, int deg) {
        poly A(f.size()), B(f.size()), C(f.size());
        A = poly_dev(f);
        B = poly_mul(f, f);
        B[0] = plus(B[0], 1);
        C = poly_inv(B, deg);
        C = poly_mul(A, C);
        C = poly_idev(C);
        return C;
    }
}

using Poly::poly;
using Poly::poly_arcsin;
using Poly::poly_arctan;

int n, m, x, k, type;
poly f, g;
char s[N];

int main()
{
    Poly::init(18);//2^21 = 2,097,152,根据题目数据多项式项数的大小自由调整，注意大小需要跟deer数组同步(21+1=22)

    read(n), read(type);

    for(int i = 0; i < n; ++ i)
        read(x), f.push_back(x);

    if(type == 0) g = poly_arcsin(f, n);
    else g = poly_arctan(f, n);

    for(int i = 0; i < n; ++ i)
        printf("%d ", g[i]);
    return 0;
}
```

他的板子跑的实在是太快了

```
/*
namespace poly_base (变换基础)
```

```
namespace poly (多项式初等函数)
namespace CDQ_NTT (分治多项式技巧)
namespace poly_evaluation (多点求值)
namespace poly_interpolation (快速插值)
namespace linear_recur (常系数线性齐次递推式 - Fiduccia)
namespace linear_recur_single (常系数线性齐次递推式 - 单点)
namespace miscellaneous (杂项)
*/
// This is a NEW polynomial template for C++11
#include <bits/stdc++.h>
#define EB emplace_back
#define lg2 std::__lg

typedef unsigned long long u64;
const int N = 530000, mod = 998244353, iv2 = (mod + 1) / 2, unity = 31;
typedef int vec[N], *pvec;
typedef std::pair <int, int> pr;
typedef std::vector <int> vector;

vec inv, fact, finv;

inline int min(const int x, const int y) {return x < y ? x : y;}
inline int max(const int x, const int y) {return x < y ? y : x;}
inline int & reduce(int &x) {return x += x >> 31 & mod;}
inline int & neg(int &x) {return x = (!x - 1) & (mod - x);}
u64 PowerMod(u64 a, int n, u64 c = 1) {for (; n; n >>= 1, a = a * a % mod) if (n & 1) c = c * a % mod; return c;}

namespace poly_base {
    int l, n; u64 iv; vec w2;

    void init(int n = N, bool dont_calc_factorials = true) {
        int i, t;
        for (inv[1] = 1, i = 2; i < n; ++i) inv[i] = u64(mod - mod / i) * inv[mod % i] % mod;
        if (!dont_calc_factorials) for (*finv = *fact = i = 1; i < n; ++i) fact[i] = (u64)fact[i - 1] * i % mod, finv[i] = (u64)finv[i - 1] * inv[i] % mod;
        t = min(n > 1 ? lg2(n - 1) : 0, 21),
        *w2 = 1, w2[1 << t] = PowerMod(unity, 1 << (21 - t));
        for (i = t; i; --i) w2[1 << (i - 1)] = (u64)w2[1 << i] * w2[1 << i] % mod;
        for (i = 1; i < n; ++i) w2[i] = (u64)w2[i & (i - 1)] * w2[i & -i] % mod;
    }

    inline void NTT_init(int len) {n = 1 << (l = len), iv = mod - (mod - 1) / n;}

    void DIF(int *a) {
        int i, *j, *k, len = n >> 1, R, *o;
        for (i = 0; i < l; ++i, len >>= 1)
            for (j = a, o = w2; j != a + n; j += len << 1, ++o)
                for (k = j; k != j + len; ++k)
                    R = (u64)*o * k[len] % mod, reduce(k[len] = *k - R), reduce(*k += R - mod);
    }

    void DIT(int *a) {
        int i, *j, *k, len = 1, R, *o;
        for (i = 0; i < l; ++i, len <<= 1)
            for (j = a, o = w2; j != a + n; j += len << 1, ++o)
                for (k = j; k != j + len; ++k)
                    reduce(R = *k + k[len] - mod), k[len] = u64(*k - k[len] + mod) * *o % mod, *k = R;
    }

```

```
61      inline void DNTT(int *a) {DIF(a);}
62      inline void IDNTT(int *a) {
63          DIT(a), std::reverse(a + 1, a + n);
64          for (int i = 0; i < n; ++i) a[i] = a[i] * iv % mod;
65      }
66
67      inline void DIF(int *a, int *b) {memcpy(b, a, n << 2), DIF(b);}
68      inline void DIT(int *a, int *b) {memcpy(b, a, n << 2), DIT(b);}
69      inline void DNTT(int *a, int *b) {memcpy(b, a, n << 2), DNTT(b);}
70      inline void IDNTT(int *a, int *b) {memcpy(b, a, n << 2), IDNTT(b);}
71  }
72
73  namespace poly {
74      using namespace poly_base;
75
76      vec B1, B2, B3, B4, B5, B6;
77
78      // Multiplication (use one buffer, 3-dft of length 2n)
79      void mul(int deg, pvec a, pvec b, pvec c) {
80          if (!deg) {*c = (u64)*a * *b % mod; return;}
81          NTT_init(lg2(deg) + 1), DNTT(a, c), DNTT(b, B1);
82          for (int i = 0; i < n; ++i) c[i] = (u64)c[i] * B1[i] % mod;
83          IDNTT(c);
84      }
85
86      // Inversion (use three buffers, 5-dft)
87      void inv(int deg, pvec a, pvec b) {
88          int i, len; assert(*a);
89          if (*b = PowerMod(*a, mod - 2), deg ≤ 1) return;
90          memset(b + 1, 0, i = 8 << lg2(deg - 1)), memset(B1, 0, i), *B1 = *a;
91
92          for (len = 0; 1 << len < deg; ++len) {
93              NTT_init(len + 1);
94
95              memcpy(B1 + (n >> 1), a + (n >> 1), n << 1), DIF(b, B2), DIF(B1, B3);
96              for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B2[i] % mod; DIT(B3);
97              for (i = n >> 1; i < n; ++i) B3[i] = B3[n - i] * iv % mod;
98
99              memset(B3, 0, n << 1), DIF(B3);
100             for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B2[i] % mod; DIT(B3);
101             for (i = n >> 1; i < n; ++i) b[i] = B3[n - i] * (mod - iv) % mod;
102         }
103     }
104
105     // Division and Modulo Operation (use five buffers)
106     void div_mod(int A, int B, pvec a, pvec b, pvec q, pvec r) {
107         if (A < B) {memcpy(r, a, (A + 1) << 2), memset(r + (A + 1), 0, (B - A) << 2); return;}
108         int Q = A - B, i, l_ = Q ? lg2(Q) + 1 : 0; NTT_init(l_);
109         for (i = 0; i ≤ Q && i ≤ B; ++i) B4[i] = b[B - i];
110         memset(B4 + i, 0, (n - i) << 2), inv(i = Q + 1, B4, B5);
111
112         std::reverse_copy(a + B, a + (A + 1), B4), NTT_init(++l_),
113         memset(B4 + i, 0, (n - i) << 2), memset(B5 + i, 0, (n - i) << 2),
114         mul(2 * Q, B4, B5, q), std::reverse(q, q + (Q + 1)),
115         memset(q + i, 0, (n - i) << 2);
116
117         if (!B) return;
118         NTT_init(lg2(2 * B - 1) + 1);
119         for (i = 0; i ≤ Q && i < B; ++i) B2[i] = b[i], B3[i] = q[i];
120         memset(B2 + i, 0, (n - i) << 2), memset(B3 + i, 0, (n - i) << 2),
```

```
        mul(2 * (B - 1), B2, B3, r), memset(r + i, 0, (n - i) << 2);
        for (i = 0; i < B; ++i) reduce(r[i] = a[i] - r[i]);
    }

    // Multiplication with std::vector (use two buffers, 3-dft)
    void mul(vector &a, vector &b, vector &ret) {
        int A = a.size() - 1, B = b.size() - 1;
        if (!(A || B)) {ret.EB((u64)a[0] * b[0] % mod); return;}
        NTT_init(lg2(A + B) + 1),
        memcpy(B1, a.data(), (A + 1) << 2), memset(B1 + (A + 1), 0, (n - A - 1) << 2),
        memcpy(B2, b.data(), (B + 1) << 2), memset(B2 + (B + 1), 0, (n - B - 1) << 2),
        DNTT(B1), DNTT(B2);
        for (int i = 0; i < n; ++i) B1[i] = (u64)B1[i] * B2[i] % mod;
        IDNTT(B1), ret.assign(B1, B1 + (A + B + 1));
    }

    // Differential
    void diff(int deg, pvec a, pvec b) {for (int i = 1; i ≤ deg; ++i) b[i - 1] = (u64)a[i] * i % mod;}

    // Integral
    void intg(int deg, pvec a, pvec b, int constant = 0) {for (int i = deg; i; --i) b[i] = (u64)a[i - 1] * ::inv[i] % mod; *b = constant;}

    // f'[x] / f[x] (use four buffers, 6.5-dft)
    void dif_quo(int deg, pvec a, pvec b) {
        assert(*a);
        if (deg ≤ 1) {*b = PowerMod(*a, mod - 2, a[1]); return;}

        int i, len = lg2(deg - 1);
        inv((deg + 1) / 2, a, B4), NTT_init(len + 1),
        memset(B4 + (n >> 1), 0, n << 1), DIF(B4, B2),

        diff(deg, a, B1), memcpy(B3, B1, n << 1),
        memset(B3 + (n >> 1), 0, n << 1), DIF(B3);

        for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B2[i] % mod;
        DIT(B3, b), *b = *b * iv % mod;
        for (i = 1; i < n >> 1; ++i) b[i] = b[n - i] * iv % mod;
        memset(b + (n >> 1), 0, n << 1);

        DIF(b, B4), DIF(a, B3);
        for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B4[i] % mod; DIT(B3);
        for (i = n >> 1; i < n; ++i) B3[i] = (B3[n - i] * iv + mod - B1[i]) % mod;

        memset(B3, 0, n << 1), DIF(B3);
        for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B2[i] % mod; DIT(B3);
        for (i = n >> 1; i < n; ++i) b[i] = B3[n - i] * (mod - iv) % mod;
    }

    // Logarithm (use DifQuo)
    inline void ln(int deg, pvec a, pvec b) {assert(*a == 1), --deg ? (dif_quo(deg, a, b), intg(deg, b, b)) : void(*b = 0);}

    // Exponentiation (use six buffers, 12-dft)
    // WARNING : this implementation of exponentiation is SLOWER than the CDQ_NTT ver.
    void exp(int deg, pvec a, pvec b) {
        int i, len; pvec c = B6; assert(!*a);
        if (*b = 1, deg ≤ 1) return;
        if (b[1] = a[1], deg == 2) return;

```

```
        memset(b + 2, 0, i = 8 << lg2(deg - 1)), memset(c, 0, i), memset(B1, 0, i),
        *c = 1, neg(c[1] = b[1]);

        for (len = 1; 1 << len < deg; ++len) {
            NTT_init(len + 1);

            DIF(c, B2), DIF(b, B3);
            for (i = 0; i < n; ++i) B4[i] = (u64)B3[i] * B2[i] % mod; DIT(B4);
            for (i = n >> 1; i < n; ++i) B4[i] = B4[n - i] * iv % mod;

            memset(B4, 0, n << 1), DIF(B4);
            for (i = 0; i < n; ++i) B4[i] = (u64)B4[i] * B2[i] % mod; DIT(B4);
            for (i = n >> 1; i < n; ++i) B4[i] = B4[n - i] * (mod - iv) % mod;

            memcpy(B4, c, n << 1), DIF(B4);
            diff(n >> 1, b, B1), DIF(B1, B5);
            for (i = 0; i < n; ++i) B4[i] = (u64)B4[i] * B5[i] % mod; DIT(B4);
            for (i = n >> 1; i < n; ++i) reduce(B5[i] = (a[i] + B4[n - i + 1] * (mod - iv) % mod *
    ::inv[i]) % mod);

            memset(B5, 0, n << 1), DIF(B5);
            for (i = 0; i < n; ++i) B5[i] = (u64)B5[i] * B3[i] % mod; DIT(B5);
            for (i = n >> 1; i < n; ++i) b[i] = B5[n - i] * iv % mod;

            if (2 << len ≥ deg) return;

            DIF(b, B3);
            for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B2[i] % mod; DIT(B3);
            for (i = n >> 1; i < n; ++i) B3[i] = B3[n - i] * iv % mod;

            memset(B3, 0, n << 1), DIF(B3);
            for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B2[i] % mod; DIT(B3);
            for (i = n >> 1; i < n; ++i) c[i] = B3[n - i] * (mod - iv) % mod;
        }
    }
}

namespace CDQ_NTT {
    using namespace poly_base;

    int lim;
    vec f, g, C1;
    int fn[N * 2], gn[N * 2];

    inline void register_g(pvec g) {for (int i = 1; 1 << (i - 1) ≤ lim; ++i) NTT_init(i), DIF(g, gn +
    (1 << i));}

    // Standard CDQ-NTT Algorithm, type `UK`
    void solve(int L, int w) {
        int i, R = L + (1 << w), M;
        if (!w) {
            // something depend on problem
            return;
        }
        solve(L, w - 1);
        if ((M = (1 << (w - 1)) + L) > lim) return;
        NTT_init(w);
        pvec ga = gn + (1 << w);
        memcpy(C1, f + L, 2 << w), memset(C1 + (1 << (w - 1)), 0, 2 << w),
        DIF(C1);
```

```
        for (i = 0; i < n; ++i) C1[i] = (u64)C1[i] * ga[i] % mod;
        DIT(C1);
        for (i = M; i < R; ++i) f[i] = (f[i] + C1[n - (i - L)] * iv) % mod;
        solve(M, w - 1);
    }

    void solve(int L, int w) {
        int i, R = L + (1 << w), M;
        if (!w) {
            // something depend on problem
            return;
        }
        solve(L, w - 1);
        if ((M = (1 << (w - 1)) + L) > lim) return;
        NTT_init(w);
        if (L) {
            pvec fa = fn + (1 << w), ga = gn + (1 << w);
            memcpy(C1, f + L, 2 << w), memset(C1 + (1 << (w - 1)), 0, 2 << w),
            memcpy(C2, g + L, 2 << w), memset(C2 + (1 << (w - 1)), 0, 2 << w),
            DIF(C1), DIF(C2);
            for (i = 0; i < n; ++i) C1[i] = ((u64)C1[i] * ga[i] + (u64)C2[i] * fa[i]) % mod;
            DIT(C1);
            for (i = M; i < R; ++i) f[i] = (f[i] + C1[n - (i - L)] * iv) % mod;
        } else {
            memcpy(C1, f, 2 << w), memset(C1 + M, 0, 2 << w),
            memcpy(C2, g, 2 << w), memset(C2 + M, 0, 2 << w),
            DIF(C1), DIF(C2),
            memcpy(fn + (1 << (w - 1)), C1, 2 << w),
            memcpy(gn + (1 << (w - 1)), C2, 2 << w);
            for (i = 0; i < n; ++i) C1[i] = (u64)C1[i] * C2[i] % mod;
            DIT(C1);
            for (i = M; i < R; ++i) f[i] = (f[i] + C1[n - i] * iv) % mod;
        }
        solve(M, w - 1);
    }
}

namespace poly_evaluation {
    using namespace poly_base;

    int cnt = 0, lc[N], rc[N];
    vec Prd_, E1, E2, E3;
    vector g[N], tmp_;

    int solve(int L, int R) {
        if (L + 1 == R) return L;
        int M = (L + R) / 2, id = cnt++, lp = solve(L, M), rp = solve(M, R);
        return poly::mul(g[lp], g[rp], g[id]), lc[id] = lp, rc[id] = rp, id;
    }

    void recursion(int id, int L, int R, const vector &poly) {
        if (L + 1 == R) return tmp_.EB(poly.back());
        int i, n = poly.size() - 1, M = (L + R) / 2, lp = lc[id], rp = rc[id],
            dl = min(n, g[lp].size() - 1), dr = min(n, g[rp].size() - 1);
        if (L + 2 == R) return
            tmp_.EB((poly[n] + (u64)poly[n - 1] * g[rp].back()) % mod),
            tmp_.EB((poly[n] + (u64)poly[n - 1] * g[lp].back()) % mod);

        vector ly, ry; ly.reserve(dl + 1), ry.reserve(dr + 1);
        NTT_init(lg2(dl + dr) + 1);
```

```
297         memcpy(E1, poly.data(), (n + 1) << 2), DIF(E1, E2), memset(E1, 0, (n + 1) << 2);
298
299         memcpy(E1, g[rp].data(), (dr + 1) << 2), DIF(E1, E3), memset(E1, 0, (dr + 1) << 2);
300         for (i = 0; i < poly::n; ++i) E3[i] = (u64)E3[i] * E2[i] % mod;
301         DIT(E3), std::reverse(E3 + 1, E3 + poly::n);
302         for (i = n - dl; i ≤ n; ++i) ly.EB(E3[i] * iv % mod);
303
304         memcpy(E1, g[lp].data(), (dl + 1) << 2), DIF(E1, E3), memset(E1, 0, (dl + 1) << 2);
305         for (i = 0; i < poly::n; ++i) E3[i] = (u64)E3[i] * E2[i] % mod;
306         DIT(E3), std::reverse(E3 + 1, E3 + poly::n);
307         for (i = n - dr; i ≤ n; ++i) ry.EB(E3[i] * iv % mod);
308
309         recursion(lp, L, M, ly), recursion(rp, M, R, ry);
310     }
311
312     vector emain(int n, pvec f, const vector &pts) {
313         int i, id, m = pts.size(), q;
314         if (!m) return vector();
315         if (!n) return vector(m, *f);
316         for (i = 0; i < m; ++i) g[i].clear(), g[i].EB(1), g[i].EB(neg(q = pts[i]));
317
318         id = solve(0, cnt = m), memcpy(Prd_, g[id].data(), (m + 1) << 2);
319         poly::inv(n + 1, Prd_, E2), memset(Prd_, 0, (m + 1) << 2);
320         if (n > 0) memset(E2 + (n + 1), 0, (poly::n - n - 1) << 2);
321
322         std::reverse_copy(f, f + (n + 1), E1), poly::mul(2 * n, E1, E2, E3),
323         memset(E1, 0, (n + 1) << 2), memset(E2, 0, (n + 1) << 2);
324
325         return tmp_.clear(), tmp_.reserve(m), recursion(id, 0, m, vector(E3 + max(n - m + 1, 0), E3 +
    (n + 1))), tmp_;
326     }
327 }
328
329 namespace poly_interpolation {
330     using namespace poly_evaluation;
331
332     vec I1, I2, I3, I4, I5;
333     pvec iresult;
334
335     void irecursion(int id, int L, int R) {
336         if (L + 1 == R) return;
337         int i, M = (L + R) / 2, lp = lc[id], rp = rc[id];
338         irecursion(lp, L, M), irecursion(rp, M, R);
339         NTT_init(lg2(R - L - 1) + 1);
340         memcpy(I1, iresult + L, (M - L) << 2), memset(I1 + (M - L), 0, (poly::n - (M - L)) << 2);
341         memcpy(I2, iresult + M, (R - M) << 2), memset(I2 + (R - M), 0, (poly::n - (R - M)) << 2);
342         memcpy(I3, g[lp].data(), (M - L + 1) << 2), memset(I3 + (M - L + 1), 0, (poly::n - (M - L + 1))
    << 2);
343         memcpy(I4, g[rp].data(), (R - M + 1) << 2), memset(I4 + (R - M + 1), 0, (poly::n - (R - M + 1))
    << 2);
344         DIF(I1), DIF(I2), DIF(I3), DIF(I4);
345         for (i = 0; i < n; ++i) I1[i] = ((u64)I1[i] * I4[i] + (u64)I2[i] * I3[i]) % mod;
346         DIT(I1), std::reverse(I1 + 1, I1 + n);
347         for (i = 0; i < R - L; ++i) iresult[L + i] = I1[i] * iv % mod;
348     }
349
350     void imain(int n, pr *pts, pvec ret) {
351         int i, id, q;
352         assert(n > 0);
353         for (i = 0; i < n; ++i) g[i].clear(), g[i].EB(1), g[i].EB(neg(q = pts[i].first));
```

```
354
355         id = solve(0, cnt = n), memcpy(Prd_, g[id].data(), (n + 1) << 2);
356         poly::inv(n, Prd_, E2), memset(Prd_, 0, n << 2);
357         if (n > 1) memset(E2 + n, 0, (poly::n - n) << 2);
358
359         for (i = 0; i < n; ++i) E1[i] = u64(n - i) * g[id][i] % mod;
360         poly::mul(2 * (n - 1), E1, E2, E3),
361         memset(E1, 0, n << 2), memset(E2, 0, n << 2);
362
363         tmp_.clear(), tmp_.reserve(n), recursion(id, 0, n, vector(E3, E3 + n));
364         for (i = 0; i < n; ++i) ret[i] = PowerMod(tmp_[i], mod - 2, pts[i].second);
365         iresult = ret, irecursion(id, 0, n), std::reverse(ret, ret + n);
366     }
367 }
368
369 namespace linear_recur {
370     using namespace poly_base;
371
372     int ld;
373     vec f, g, L1, L2, L3;
374
375     void __builtin_divmod(int d) {
376         int i; NTT_init(ld);
377         std::reverse_copy(f + d, f + 2 * d, L3);
378         memset(L3 + d, 0, (n - d) << 2);
379         DNTT(L3);
380         for (i = 0; i < n; ++i) L3[i] = (u64)L3[i] * L2[i] % mod;
381         IDNTT(L3);
382         std::reverse(L3, L3 + d), memset(L3 + d, 0, (n - d) << 2);
383         DNTT(L3);
384         for (i = 0; i < n; ++i) L3[i] = (u64)L3[i] * L1[i] % mod;
385         IDNTT(L3);
386         for (i = 0; i < d; ++i) reduce(f[i] -= L3[i]);
387         memset(f + d, 0, d << 2);
388     }
389
390     void solve(int Q, int d) {
391         int i, df = 1, z = lg2(Q); bool alive = false;
392         assert(d > 0), ld = lg2(2 * d - 1) + 1;
393         if (d == 1) alive = true, *f = 1;
394
395         NTT_init(ld),
396         std::reverse_copy(g + 1, g + (d + 1), L1),
397         memset(L1 + d, 0, (n - d) << 2),
398         poly::inv(d, L1, L2);
399
400         NTT_init(ld),
401         memset(L2 + d, 0, (n - d) << 2),
402         memcpy(L1, g, d << 2),
403         memset(L1 + d, 0, (n - d) << 2),
404         DIF(L1), DIF(L2);
405
406         for (; --z ≥ 0; ) {
407             df = df << 1 | (Q >> z & 1);
408             if (df < d) continue;
409             if (!alive) alive = true, f[df >> 1] = 1;
410             NTT_init(ld), DNTT(f);
411             for (i = 0; i < n; ++i) f[i] = (u64)f[i] * f[i] % mod;
412             IDNTT(f);
413             if (df & 1) std::copy_backward(f, f + (2 * d - 1), f + 2 * d), *f = 0;
```

```
            __builtin_divmod(d);
        }
    }
}

namespace linear_recur_single {
    using namespace poly;

    int ld;
    vec f, g, L1, L2, L3, L4;

    int get(int Q, int d) {
        if (Q < d) return f[Q];

        int i, j, ret = 0, halves = 0;
        assert(d > 0), ld = lg2(2 * d - 1) + 1,

        mul(2 * d - 1, f, g, L1), memset(L1 + d, 0, (n - d) << 2),
        memcpy(L2, g, (d + 1) << 2), memset(L2 + (d + 1), 0, (n - d - 1) << 2);

        for (; Q ≥ d; Q >>= 1) {
            NTT_init(ld), DIF(L1, L4), DIF(L2, L3),
            NTT_init(ld - 1), ++halves;
            if (Q & 1) {
                for (j = i = 0; i < n; ++i, j += 2)
                    L4[i] = ((u64)L4[j] * L3[j + 1] + u64(mod - L3[j]) * L4[j + 1]) % mod;
                for (i = 0; i < n; ++i) L4[i] = (u64)L4[i] * w2[i] % mod;
                IDNTT(L4), L4[n] = *L4, memcpy(L1, L4 + 1, d << 2);
            } else {
                for (j = i = 0; i < n; ++i, j += 2)
                    L4[i] = ((u64)L4[j] * L3[j + 1] + (u64)L3[j] * L4[j + 1]) % mod;
                IDNTT(L4, L1);
            }
            for (j = i = 0; i < n; ++i, j += 2) L3[i] = (u64)L3[j] * L3[j + 1] % mod;
            IDNTT(L3, L2);
            if (*L2 ≠ 1) L2[d] = (*L2 ? *L2 : mod) - 1, *L2 = 1;
        }

        poly::inv(Q + 1, L2, L3);
        for (i = 0; i ≤ Q; ++i) ret = (ret + (u64)L1[i] * L3[Q - i]) % mod;
        return PowerMod(iv2, halves, ret);
    }
}

namespace miscellaneous {
    using namespace poly_base;

    vec M1, M2, M3, M4;
    vec Ma[200];

    // calculate b(x) = a(x + c), poly_base::init need 'false'
    void translate(int deg, pvec a, int c, pvec b) {
        if (!deg) {*b = *a; return;}
        int i; u64 pc = 1; NTT_init(lg2(deg) + 2);
        for (i = 0; i ≤ deg; ++i) M1[i] = (u64)a[i] * fact[i] % mod;
        memset(M1 + i, 0, (n - i) << 2);
        for (i = deg; i ≥ 0; --i, pc = pc * c % mod) M2[i] = finv[deg - i] * pc % mod;
        DIF(M1), DIF(M2);
        for (i = 0; i < n; ++i) M1[i] = (u64)M1[i] * M2[i] % mod;
        DIT(M1);
```

```
        for (i = 0; i ≤ deg; ++i) b[i] = M1[n - deg - i] * iv % mod * finv[i] % mod;
    }

    // compute a(1), a(z), a(z^2), ... , a(z^(len-1)) to b
    void czt(int deg, int len, int z, pvec a, pvec b) {
        int i; u64 iz = PowerMod(z, mod - 2);
        if (!len) return;
        if (!deg || !z) return std::fill(b, b + len, *a);
        NTT_init(lg2(deg + len - 1) + 1);
        M1[1] = *M1 = M3[1] = *M3 = 1;
        for (i = 2; i < deg + len; ++i) M1[i] = M1[i - 1] * (u64)z % mod, M3[i] = M3[i - 1] * iz % mod;
        for (i = 2; i < deg + len; ++i) M1[i] = (u64)M1[i] * M1[i - 1] % mod, M3[i] = (u64)M3[i] * M3[i
- 1] % mod;
        memset(M1 + i, 0, (n - i) << 2);
        for (i = 0; i ≤ deg; ++i) M2[deg - i] = (u64)M3[i] * a[i] % mod;
        memset(M2 + i, 0, (n - i) << 2),
        DIF(M1), DIF(M2);
        for (i = 0; i < n; ++i) M1[i] = (u64)M1[i] * M2[i] % mod;
        DIT(M1);
        for (i = 0; i < len; ++i) b[i] = M1[n - deg - i] * iv % mod * M3[i] % mod;
    }

    inline int __builtin_inner(int len, pvec a, pvec b) {
        int i = 0; u64 ans = 0;
        #define term(k) (u64)a[i + k] * b[i + k]
        for (; i + 15 < len; i += 16) {
            ans = (ans + term(0) + term(1) + term(2) + term(3)
                       + term(4) + term(5) + term(6) + term(7)
                       + term(8) + term(9) + term(10) + term(11)
                       + term(12) + term(13) + term(14) + term(15)
                ) % mod;
        }
        #undef term
        for (; i < len; ++i) ans += (u64)a[i] * b[i];
        return ans % mod;
    }

    // compositional inverse
    void cinv(int deg, pvec a, pvec b) {
        assert(!*a), *b = 0;
        if (--deg ≤ 0) return;
        int i, j, k, d, B, *c;

        NTT_init(lg2(2 * deg - 1)),
        memcpy(M1, a + 1, deg << 2), memset(M1 + deg, 0, (n - deg) << 2),
        poly::inv(deg, M1, M2),

        NTT_init(lg2(2 * deg - 1) + 1),
        memset(M2 + deg, 0, (n - deg) << 2),

        DIF(M2, M1), B = sqrt(deg),
        memset(*Ma, 0, n << 2), **Ma = 1;
        for (j = 1; j ≤ B; ++j) {
            DIF(Ma[j - 1], M3), c = Ma[j];
            for (i = 0; i < n; ++i) M3[i] = (u64)M3[i] * M1[i] % mod;
            DIT(M3), *c = *M3 * iv % mod;
            for (i = 1; i < deg; ++i) c[i] = M3[n - i] * iv % mod;
            memset(c + i, 0, (n - i) << 2);
        }
        DIF(Ma[B], M1), memset(M3, 0, n << 2), *M3 = M4[deg - 1] = 1;
```

```
533
534         for (d = 0; ; ) {
535             for (k = 0; k < B && d < deg; ++k, ++d)
536                 b[d + 1] = (u64)__builtin_inner(d + 1, Ma[k + 1], M4 + (deg - d - 1)) * inv[d + 1] %
    mod;
537             if (d ≥ deg) break;
538             DIF(M3);
539             for (i = 0; i < n; ++i) M3[i] = (u64)M3[i] * M1[i] % mod;
540             DIT(M3), *M3 = *M3 * iv % mod;
541             for (i = 1; i < deg; ++i) M3[i] = M3[n - i] * iv % mod;
542             memset(M3 + i, 0, (n - i) << 2),
543             std::reverse_copy(M3, M3 + deg, M4);
544         }
545     }
546 }
547
```

参考资料

1. 学习笔记：多项式全家桶 By NaCly_Fish %鱼
2. 多项式全家桶
3. FFT什么的 By YWW
4. 【Learning】 多项式全家桶（加减乘除逆元ln exp）ByRogerDTZ
5. OI Wiki _多项式 OI Wiki yyds!
6. [[学习笔记]多项式相关运算2(求导/积分,牛顿迭代,开根,求ln/exp,快速幂,拉格朗日反演)](https://blog.csdn.net/qq_34940287/article/details/104318913?utm_medium=distribute.pc_relevant.none-task-blog-OPENSEARCH-2.control&depth_1-utm_source=distribute.pc_relevant.none-task-blog-OPENSEARCH-2.control)
7. 浅谈多项式 by CYJian